

Master's Thesis

Development of a Source Library for Transfer Learning: Leveraging Clustering and Contrastive Learning Techniques

Sneha Banerjee
Matriculation Number: 430069

Monday 29th April, 2024

1st Examiner:	Prof. Dr. Christian Bauckhage, University of Bonn
2nd Examiner:	Prof. Dr.-Ing. Alexander Verl, ISW, University of Stuttgart
Supervisor:	Daniel Schiller

Declaration of Originality

Name	Sneha Banerjee
Matriculation number	430069
Address	Allmandring 8A,70569 Stuttgart
Title	<i>Development of a Source Library for Transfer Learning: Leveraging Clustering and Contrastive Learning Techniques</i>

I now declare,

- that I wrote this work independently,
- that no sources other than those stated are used and that all statements taken from other works—directly or figuratively—are marked as such,
- that the work submitted was not the subject of any other examination procedure, either in its entirety or in substantial parts,
- that I have not published the work in whole or in part, and
- that my work does not violate any rights of third parties and that I exempt the University against any claims of third parties.

Abstract

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Task and Goal	1
1.3	Approach	1
1.4	Contribution	1
2	Related Work	2
3	Fundamentals	3
3.1	Bin Picking	3
3.1.1	Definition and Application	3
3.1.2	Gripper Types	3
3.1.3	Model-Based Approaches	3
3.1.4	Model-Free Approaches	3
3.1.5	Robot System	3
3.2	Deep learning-based 3D Image processing	3
3.2.1	Convolution Neural Network	3
3.2.2	Autoencoders	4
3.2.3	Siamese Network	6
3.3	Clustering Algorithms	8
3.3.1	Density-based Clustering	9
3.3.2	Distribution-based Clustering	9
3.3.3	Hierarchical-based Clustering	10
3.4	Evaluation metrics for Clustering Algorithms	10
3.4.1	Silhouette Score	10
3.4.2	Calinski-Harabasz Index	10
3.4.3	David-Bouldin index	11
3.4.4	Density Based Cluster Validation Index	12
3.5	Transfer learning	16
3.5.1	Definition	16
3.5.2	Categories	17
3.5.3	Relevant Algorithms	18
3.6	Optimal Transport Problem	18
3.7	Sinkhorn-Knopp Algorithm	18
3.8	Dimensionality reduction	18
3.9	Network generation process with PQ-Net++	18
4	Method	19
4.1	Architecture	19
4.2	Encoder Backbone	20
4.2.1	PointNet Autoencoder	20
4.2.2	Dynamic Graph Convolutional Neural Network	22
4.3	Point-level Clustering Module	25
4.3.1	Class Probability prediction	26
4.3.2	Label reassigning	26
4.4	Instance-level Contrasting Module	28
4.5	Loss function	29
4.6	Clustering Algorithm	29
4.7	Suitable Algorithm and Adjustments	30
4.8	Whatever new Concept we propose	31
4.9	Source and Target Objects	31
4.10	Data Generation	31
4.11	Important Aspects	31

5	Application of algorithms	32
6	Experiments	33
7	Implementation in a Bin Picking Cell	34
8	Conclusions	35
8.1	Discussions	35
8.2	Limitations	35
8.3	Future Scope	35
9	Summary	36
	List of Acronyms	i
	List of Figures	ii
	List of Tables	iii
	List of Symbols	iv
	Bibliography	v

1 Introduction

1.1 Motivation

1.2 Task and Goal

1.3 Approach

1.4 Contribution

2 Related Work

3 Fundamentals

3.1 Bin Picking

3.1.1 Definition and Application

3.1.2 Gripper Types

3.1.3 Model-Based Approaches

3.1.4 Model-Free Approaches

3.1.5 Robot System

3.2 Deep learning-based 3D Image processing

3.2.1 Convolution Neural Network

Inspired from the visual cortex of humans, convolutional neural network(Convolutional Neural Network (CNN) or ConvNet) is a type of deep neural network designed for processing data which appear in a grid like manner like images, videos, etc. It was introduced by Yann LeCun et. al in [32] in 1998. It gets its name because of the usage of a special kind of linear mathematical operation called the convolution instead of using matrix multiplication as prevalent in the pre-existing neural networks. The key components of a CNN are - Convolution layer, activation function, pooling layer, loss function, output layer. The principle building block of a CNN is the convolution layer. It consists of a number of learnable filters (kernels) which can be visualised like a cubic block. The success of CNNs can be attributed to three major concepts: sparse interactions, parameter sharing and equivariant representations.

Sparse interactions In traditional fully connected network, matrix multiplication is performed which involves a parameter matrix. The interaction between input and output units are captured by a distinct parameter in the parameter matrix. But CNNs have sparse interactions, i.e. only a subset of units or neurons in a layer is connected to a local region in the preceding layer. This is done by using kernels that are significantly smaller than the input. This implies, less number of parameters need to be stored which reduces the memory consumption and also it is computationally efficient because it has to perform fewer operations. For example, if there are m inputs and n outputs, the fully connected network would need to store $m \times n$ parameters and have a runtime of $O(m \times n)$ time complexity per input. On the other hand, if we restrict the number of connections for each unit to be k , then we would have $k \times n$ and have a runtime of $O(k \times n)$ time complexity per input, where k is quite some fold lesser in magnitude as compared to m . Moreover, since convolution layers are stacked upon one another in a deep convolution network, units in the deeper layers have a larger receptive field, because of its indirect interaction with a larger region in the input.

Parameter sharing Another important focus behind using the convolution layer was to reduce the number of parameters in a Fully Connected Network (FCN). For example in a 1024×1024 image, a FCN would have over 1 million hidden units, which means it would have over 1 trillion trainable parameters. But the pixels in an image are only locally correlated. So, CNNs make use of the kernels to limit the focus on smaller regions on the image at a time known as the receptive field. This significantly reduces the number of trainable parameters, thus reducing the memory consumption. [9]. These layers perform a convolution operation between the input (eg. image) I and the kernel K to produce an output S known as the feature map as in Eq. 1.

Equivariant Representations A function is said to be equivariant if the input to a function is changed, then the output changes in the similar way. Because of the parameter sharing mechanism, convolutions operations are translation equivariant. When the kernels are applied on an input image, the convolution layers generate a 2D map of where the particular feature occurs in the particular image. Furthermore, a pixel is related to its neighboring pixels to form a meaningful context(create a feature e.g. an edge in an image) but it is not limited to where it can occur throughout the image. Thus it creates multiple filters

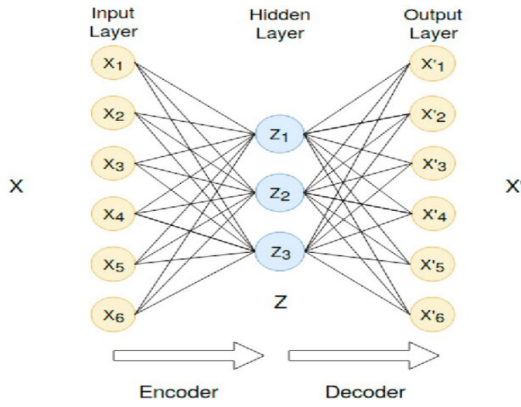


Figure 3.2.1: Architecture of an undercomplete autoencoder.[37]

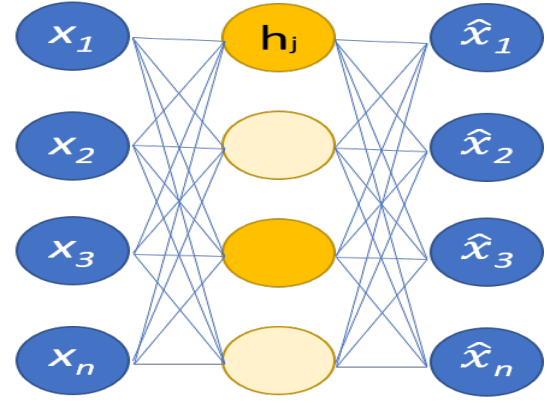


Figure 3.2.2: Architecture of a single layer sparse encoder. The hidden nodes in bright yellow are active, while the ones in light yellow are set to zero, hence inactive[3]

each of which look for the same feature throughout the image. But it is also to be kept in mind that convolution is not equivariant to other geometric and affine transformations like rotation, scaling, etc. Since convolution operations are commutative in nature, Eq. 1 can also be written as Eq. 2. Typically, Eq. 2 is easier to incorporate into a machine learning library since values for both 'm' and 'n' varies within a small range.[17]

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m,j-n) \quad (1)$$

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i-m,j-n)K(m,n) \quad (2)$$

Activation function The next step in a CNN is to apply an activation function. the purpose of using a non linear activation function is to capture the non-linearity in the data. Moreover if non-linearity is not used in between the multiple layers of a neural network, the network is effectively only one layer deep which is not capable even to capture the non-linearity in real world datasets. Rectified Linear Unit (ReLU) is the most commonly used non-linear activation function used in CNNs because unlike sigmoid function or tanh function it does not penalise "too correct" data points. Another remarkable benefit of using ReLU is it's ability to propagate gradient through deep networks with a constant factor. Also it is more memory efficient to use ReLU as it doesn't require to store the ReLU outputs separately as compared to tanh outputs.

Pooling The third step of a CNN is to use the pooling layer. The main purpose of this operation is to make the detection of the features robust to the exact location of the eye (i.e. invariant to small translations). The different types of pooling operations are - max pooling and average pooling. It also helps in reducing the dimensionality of the input without losing too much information. This is done to make the computations faster down the deeper layers of the network. If the pooling operations are performed after every k pixels, then the next layer processes inputs that are k times lesser. Since the number of parameters in a layer are dependent on the size of the input to the layer, it significantly reduces the computational overhead on using pooling operations.

Complete CNN

3.2.2 Autoencoders

Autoencoders are a special type of feedforward neural network in which the output tries to reconstruct the input. It is predominantly used in unsupervised learning for the tasks of dimensionality reduction, learning feature representations, and for data compression. It tries to encode the input data into a more compact representation with lower dimensions called the "code"[17] or "latent space". The idea is that this latent space representation should capture the most vital aspects of the input data. This is done by

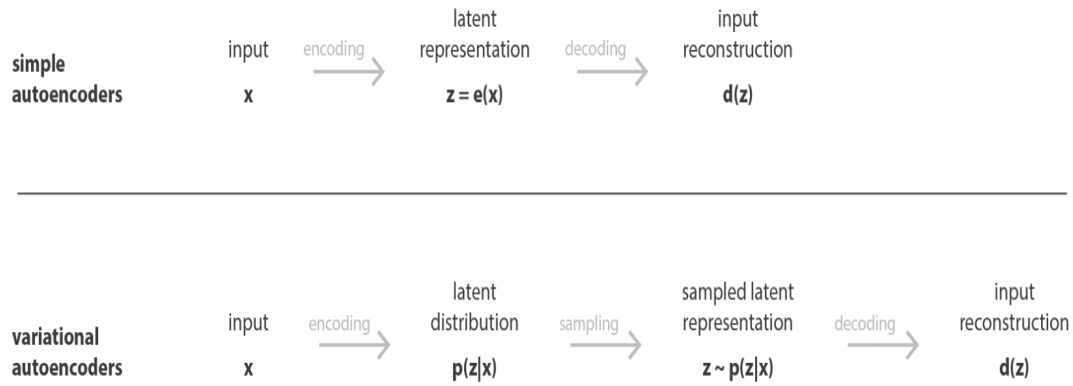


Figure 3.2.3: Difference between a vanilla autoencoder and a Variational Autoencoders (VAE).[51]

the first part of the network called the encoder. The second part of the network, the decoder then tries to decode this compressed representation of the input data to reconstruct the original input data as accurately as possible. during the training of an autoencoder, the goal is to minimise this reconstruction loss, i.e. the difference between the original input data and the output of the decoder. There are different types of autoencoders - undercomplete autoencoders, convolutional autoencoders, regularized autoencoders, and variational autoencoders,

Undercomplete autoencoders ensures that the dimension of the latent space representation is less than the dimension of the original input data as show in Fig.3.2.1. Because, the output of the decoder to be the exact copy of the input is of no use. Rather, if the dimension of the code is less than that of the input, then it ensures that the autoencoder learns those representative features of the input data which are most salient[17]. Convolutional autoencoders are an extension of traditional autoencoders where convolution layers are used as building blocks in both the encoder and the decoder part of the autoencoder. After training the network, the encoder part is used for extracting the features of the data and the decoder part is used for the reconstruction of the input data.

Regularized autoencoders are a special type of autoencoders that employs one or more regularisation techniques to prevent the model from overfitting. The different types of regularisations used are - L1 or L2 Regularisation which adds a penalty to the loss function depending on L1 or L2 norm of the model weights As a result of this, the model is capable of learning sparse representation of the training data where many weights are set to a very small value or zero as shown in Fig3.2.2. Dropout is also used as a regularization technique where a percentage of the model's nodes are set to zero during training so that the model doesn't rely heavily on any the weight of any particular node, thus preventing overfitting and improving generalisation on unseen data. The previously mentioned regularisation techniques give rise to a variation of regularised autoencoders called the sparse autpencoders[39]. Sometimes noise is also added to the training data or the hidden layers of the model to increase the robustness the model giving rise to denoising autoencoders[53]. One more regularisation technique if to apply a contractive regularization term based on Frobenius norm of the Jacobian matrix of the model's hidden layer activations with respect to the input data[46, 3]. This ensures that a small neighborhood of the input data corresponds to a small neighborhood in the latent space representation, which means small perturbations in the input data leads to small or zero variation in the latent space representation[46, 3]. By doing so, it makes the model more robust to small changes in the input data, thus preventing overfitting.

VAEs are a distinct type of autoencoders which produces latent space representations that are continuous, which allows random sampling and interpolation for the generation of new datapoints. The difference between a vanilla autoencoder and a VAE is shown in Fig.3.2.3. Instead of producing a single vector for the latent space representation, it generates two vectors: a vector of means μ and a vector of standard deviations γ for all datapoints. An encoding is then sampled from a distribution with mean μ and standard deviation

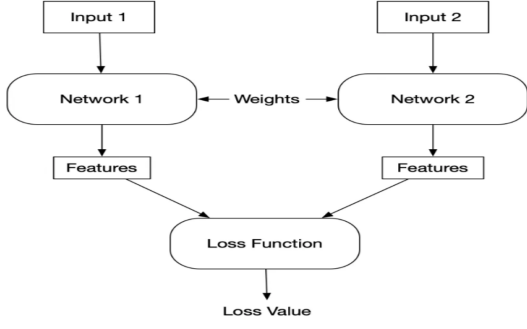


Figure 3.2.4: A generic siamese network.[1]

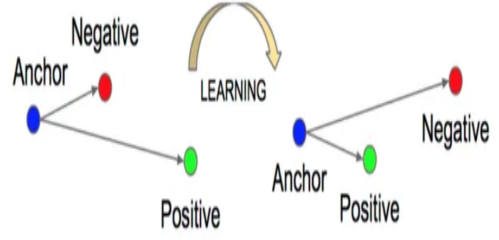


Figure 3.2.5: Before (left) and after (right) minimizing triplet loss function[50]

γ . Thus even for the same input, i.e. when the mean and the standard deviation are the same, the sampled encoding would vary because of the involvement of the sampling procedure. The mean vector controls the position where the encoding of the input should have its centre and the standard deviation controls the area over which the sampled encoding is allowed to vary from the mean. As a result of this, the decoder learns to decode not just the encoding of a single point but also the points in the neighborhood and hence a continuous latent space representation is obtained. In order to ensure that the latent space representation satisfies that the nearby encoding are similar to each other on a local scale while also facilitating interpolation on a global scale VAEs jointly optimize the reconstruction loss and the Kullback–Leibler divergence (KL)[31] loss.[28, 24]

3.2.3 Siamese Network

Traditional deep learning models have shown ground breaking results in a multitude of applications like image recognition and classification, web scraping, speech recognition and caption generation. But it has a limitation that the models tend to perform well if the training and testing data points are from the same distribution. If the model is to be made capable to predict datapoints belonging to an unknown class, we need to re-train the model with an abundant number of training samples from that class and then predict it with unseen samples of that class. This technique can often be computationally expensive and can increase exponentially if the number of classes increases. Also generating a huge number of samples for a new class of objects can be tedious and quite challenging in multiple scenarios. The Siamese network[29, 5] is one such way to mitigate the above mentioned problem. The idea behind siamese networks is that two identical copies of the neural neural network with shared weights process two distinct datapoints as shown in Fig.3.2.4. Because the networks have shared parameters, it ensures that two very similar datapoints cannot have too different locations in the feature maps as both the networks evaluate the same function. In siamese networks, the model learns a similarity function, the loss function, which plays the pivotal role in determining how similar or dissimilar the datapoints are to one another. The two main loss functions used in siamese networks are contrastive loss function[5] and triplet loss function[4]. Since the goal of a siamese network is not to perform classification task on datapoints, rather to compute the similarity between them, contrastive loss functions are more suited for this task as compared to cross-entropy loss functions, as it is for classification tasks.

Contrastive loss function evaluates how capable a siamese network is in deciding the similarity between two given datapoints as is given by Eq. 3[1]

$$\mathcal{L}_{\text{con}}(Y, D_w) = (1 - Y) \frac{1}{2} (D_w)^2 + (Y) \frac{1}{2} \max(0, m - D_w)^2 \quad (3)$$

where $\mathcal{L}_{\text{con}}$ is the calculated contrastive loss, D_w is the Euclidean distance between the outputs of the twin networks[1]. Y takes the value 0 or 1. If the two datapoints belong to the same class then Y takes the value 0, otherwise it takes the value 1[1]. $\max()$ is a function to choose the higher of the two values, 0 or $m - D_w$ where m is the margin and has a value greater than 0[1]. The usage of a margin in this equation ensures

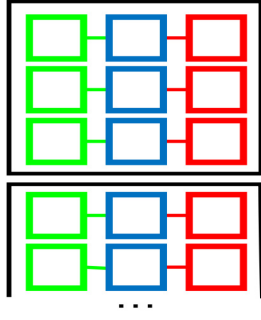


Figure 3.2.6: Minibatches in offline triplet mining. The green boxes are positive datapoints, the blue boxes are anchor datapoints and the red boxes are negative datapoints.

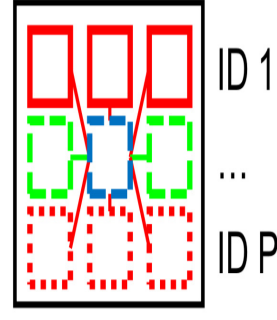


Figure 3.2.7: Triplets constructed only within the minibatch. There are P object classes in the minibatch. The green boxes are positive datapoints, the blue box is the anchor datapoint and the red boxes are negative datapoints.

that the pair of datapoints which the dissimilar beyond this margin value are excluded while calculating the loss function[1]. The euclidean distance between the embeddings of the two datapoints are given by the Eq. 4[1]

$$D_w(X_1, X_2) = \sqrt{\{G_w(X_1) - G_w(X_2)\}^2} \quad (4)$$

where $G_w(X_i)$ is the output of the twin network such that $i \in \{1, 2\}$ as show in Fig.3.2.4.

Triplet loss function[57] trains the siamese network in a way that a triplet of datapoints are presented to the siamese network. by applying triplet loss the model tries to group the datapoints similar to the anchor datapoint, close to one another while increasing the distance between the anchor datapoint and the datapoints dissimilar to it as shown in Fig.3.2.5. In order to do so, a positive value called margin is used which increases the distance between dissimilar datapoints and also eliminates the output of any trivial solution. Euclidean distance or cosine distance is used as the distance metric while calculating the triplet loss and shown in Eq. 5[21, 1]

$$\mathcal{L}_{\text{tri}}(a, p, n) = \sum_{\substack{a, p, n \\ y_a = y_p \neq y_n}} \max(0, d(a, p) - d(a, n) + \text{margin}) \quad (5)$$

where y_x is the class label for the datapoint x for $x \in \{a, p, n\}$, a is the anchor datapoint, p is the positive example having the same class label as the anchor datapoint and n is the negative example having a different class label as compared to the anchor datapoint. but there are some practical problems related to the use of triplet loss function. The number of possible triplets grow cubically with the increase in the size of the dataset. Also most of the triplets are often uninformative. Thus mining "hard triplets" or the ones crucial for the learning of the model becomes a challenging task. As per the definition of the triplet loss function, the triplets can be of three types: easy triplets, hard triplets and medium hard triplets. The triplets which satisfy the condition $d(a, p) + \text{margin} < d(a, n)$ results to the loss value being 0, hence are called the easy triplets.[50] For the triplet for which the negative datapoint is closer to the anchor as compared to the positive datapoint i.e. $d(a, n) < d(a, p)$ are the examples of hard triplets.[50] And the medium hard triplets are the ones where the negative datapoints are not closer to the anchor datapoint as compared to the positive datapoint but still results in positive loss i.e. $d(a, p) < d(a, n) < d(a, p) + \text{margin}$.[50] Thus the medium hard triplets are the most essential for the model training as they constitute for the most useful information. Two triplet mining procedures are used to mitigate the issue of excessive computational overhead with increase in dataset size. They are offline triplet mining and online triplet mining. In offline hard triplet mining, at first the dataset is manually processed to find hard triplets and then they are used for training the model. But manually finding the hard triplets can be quite tedious. Online triplet mining is a better approach as compared to the one mentioned above. The model is trained in minibatches of the dataset as shown in Fig.3.2.6. Using only the triplets that were mined from the dataset could be a wasteful design

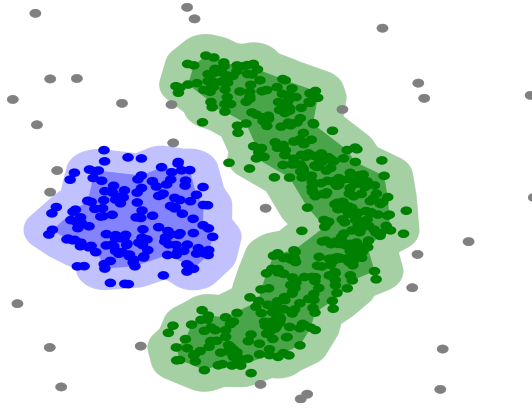


Figure 3.3.1: Density-Based Spatial Clustering of Applications with Noise (DBSCAN) can find non-linearly separable non-convex clusters.[13]

choice in such scenario. So in a minibatch, it contains way more potential triplets than the ones that were mined. So each member of another triplet becomes an additional negative datapoint example. But both hard positive and hard negative datapoints are required for training. An even better design to mitigate that would be to choose let's say K datapoints from P different classes in the dataset, where the K positive examples serve as hard positives while the rest of the datapoints in the minibatch serve as hard negatives as shown in Fig.3.2.7[21].

Thus siamese networks are very robust to the class imbalance in dataset which is very common in real datasets. Since it depends on very few datapoints of a particular object class, it has good generalisation capabilities for unseen datapoints in the future. They are also very robust to small perturbations in the datapoints due to difference in lighting conditions or orientation or background, as they keep similar objects close to each other even if the datapoints are slightly different. Since the siamese networks places similar datapoints together while increasing the distance between dissimilar datapoints, they are often useful for learning semantic similarity within the datapoints. However despite its benefit, training a siamese network takes longer as compared to traditional neural networks because it involves learning from quadratic pair of datapoints.[1]

3.3 Clustering Algorithms

When the dataset does not have any labelled data, then it is said to be unsupervised learning. In this case, there is no ground truth available to measure the correctness of the outputs generated by the Machine Learning (ML) models. The primary focus of unsupervised learning is to find hidden and interesting pattern in the data. Unsupervised learning is of utmost importance in the Artificial Intelligence (AI) world as several real world datasets do not have available annotations, which requires a lot of human effort. Unsupervised learning algorithms can be broadly categorized into the following domains-clustering, dimensionality reduction, and association analysis. Clustering algorithms aim at grouping unlabelled data into groups or clusters based on how similar or dissimilar the datapoints are to one another. It can reveal underlying hidden pattern in the data and is used in applications like image segmentation, fraud detection, etc. Dimensionality reduction reduces the number of irrelevant features or dimensions of the dataset. The inclusion of more features does help in the better representation of the dataset but it also significantly increases the memory consumption and complexity to work with it. Also it is often difficult to visualise real-life datasets with too many features. Association analysis is a rule-based unsupervised learning method that reveals the relationship between attributes in the dataset. It is used in applications like market analysis, intrusion detection, etc. Clustering algorithms can be broadly classified into the following categories: density-based, distribution-based, and hierarchical-based.

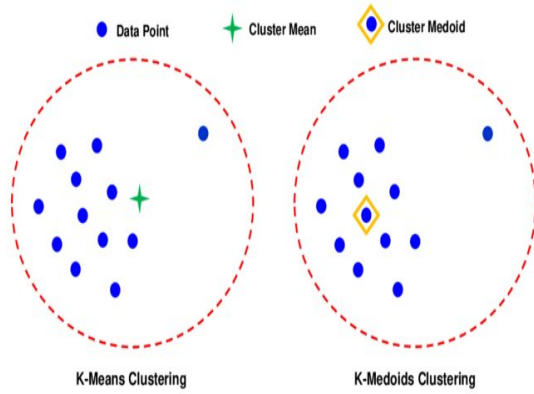


Figure 3.3.2: Difference between K-Means and K-Medoids.[15]

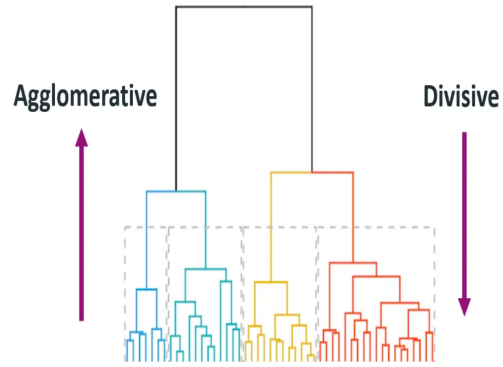


Figure 3.3.3: A dendrogram in hierarchical clustering.[22]

3.3.1 Density-based Clustering

In density-based clustering, the algorithm looks for areas of high concentration of the datapoints and groups them as a cluster. The benefit of this algorithm is that the shape a cluster can be is not limited and hence doesn't necessarily have to be convex in nature as shown in Fig. 3.3.1. Moreover, these algorithms are also very robust to outliers as they do not force the outliers to belong to any category but are rather ignored as it is unlikely that outliers can form an area of high concentration of datapoints. I have carried on the experiments on this thesis on two such density-based clustering algorithms called DBSCAN[16] and Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) algorithms[35]. The available scikit-learn implementations were used for this purpose[42]. The DBSCAN[16] algorithm does not require the users to define the number of clusters to be generated which doesn't force dissimilar datapoints to belong to the same cluster. However, this algorithm was still sensitive to two parameters ϵ , the maximum distance between the datapoints to be considered in the same cluster and the minimum number of samples in the cluster which the user needs to define [42]. But finding an optimum value for these parameters often require domain expertise and are dependent on the data. The HDBSCAN[35] algorithm mitigates this issue and thus does not require the user to define these two parameters. It is a hierarchical density-based clustering that performs the DBSCAN algorithm over multiple ϵ values to find the most stable result.

3.3.2 Distribution-based Clustering

In distribution-based algorithms, a datapoint is said to be a member of the cluster depending on the probability of its membership to the cluster. The more the distance of a point increases from the centre of the cluster, the less is its probability of belonging to that cluster. Centroid-based algorithms group the datapoints based on some initial cluster centres. Once all the datapoints are softly assigned to some cluster membership, the cluster centres are recalculated and this process is iterated until convergence. These algorithms are sensitive to the initial parameters like the cluster centres chosen in the first step. Another major disadvantage of these clustering algorithms is that they always form spherical clusters. The user also needs to define the number of clusters the dataset is to be grouped in, which makes it sensitive to outliers. However, these algorithms can be executed very fast and we have used two such algorithms in our experiments- K-Means[26] and K-Medoids[27]. In K-Means, the mean of the datapoints of the clusters is assigned as the cluster-centroid. It might not be an actual datapoint in the dataset, rather a blurred, noisy average of a datapoints in the cluster. On the contrary, the K-Medoids algorithm assigns an actual datapoint of the dataset, that is most centrally located as the cluster centroid as shown in Fig.3.3.2. Thus K-Medoids is more robust to outliers and noises as compared to K-Means.

3.3.3 Hierarchical-based Clustering

Hierarchical-based clustering algorithm that form a hierarchy of clusters. Datapoints in a cluster are more similar to each other as compared to other groups. this hierarchy of clusters is visualised by a hierarchy tree called dendograms. hierarchical clustering algorithms can be of two types - agglomerative and divisive. In agglomerative clustering, each datapoint is considered as a separate cluster in the first step. Then these clusters are merged into one another until only one cluster remains. Thus at the end, the last level cluster consists of all the datapoints in the dataset. The divisive method is the reverse procedure of the agglomerative method. In the beginning, all the datapoints are considered to be in a single cluster and gradually the cluster is broken into smaller clusters, until each cluster consists of only one datapoint. A visual representation of a dendogram has been shown in Fig.3.3.3

3.4 Evaluation metrics for Clustering Algorithms

The ground truth or labels are available in supervised learning. So to determine the quality of the clustering algorithm we can calculate the level of accuracy by comparing the labels predicted by a model to the ground truth. But most real world datasets do not have ground truth. Therefore, it makes it challenging in unsupervised learning paradigm to determine the quality of the clustering algorithm. But a good clustering algorithm is still expected to have little difference between the objects in a cluster i.e. small within-cluster variance and more difference between objects from different clusters i.e. large between-cluster variance. Thus the evaluation metrics used in the literature could be categorised into two broad groups: extrinsic measures and intrinsic measures, depending on the availability of the ground truth. Extrinsic measures, when the ground truth is available uses metrics like Rand Index, Mutual Information, V-measure and Fowlkes-Mallows Scores. Intrinsic measures, when the ground truth is not available mostly uses Silhouette Score, Calinski Harabaz Index and David Bouldin index.

3.4.1 Silhouette Score

Silhouette Score is used to calculate the distance between points within a cluster as compared to its distance from the points in the neighbouring cluster and is given by Eq. 6

$$S_c = \frac{(n_c - i_c)}{\max(n_c, i_c)} \quad (6)$$

where S_c is the Silhouette Coefficient, n_c is the average of the nearest-cluster distance of each datapoint and i_c is the average distance between the datapoint within the cluster. The Silhouette coefficient lies within the range $[-1, 1]$, where the more closer the coefficient is to 1, the better is the clustering results. When the Silhouette Coefficient is close to 0, it implies that the datapoints are on or very close to the decision boundary between adjacent clusters, whereas negative values imply that the datapoints have been wrongly classified. But the drawbacks of using this metric is that it tends to favour dense and convex clusters. Since it takes into account, the average of the distance within a cluster as well as between adjacent clusters, it doesn't perform well if different clusters have a significant difference in their respective sizes, which is quite a common phenomenon in real world datasets. The chosen distance metric for the calculation of the Silhouette score also has a significant effect on it and can lead to very different results.

3.4.2 Calinski-Harabasz Index

Calinski-Harabasz Index (C-H Index) or the Variance Ratio Criterion is used to measure the dispersion between the datapoints within a cluster as compared to in-between clusters as in given by Eq. 7

$$S_{CH} = \frac{tr(B_k)}{tr(W_k)} \times \frac{(n_E - k)}{k - 1} \quad (7)$$

where S_{CH} is the C-H Index, $tr(B_k)$ is the trace of the in-between cluster dispersion matrix as in Eq. 8 and $tr(W_k)$ is the trace of the within-cluster dispersion matrix as in Eq. 9

$$B_k = \sum_{q=1}^k n_q (c_q - c_o)(c_q - c_o)^T \quad (8)$$

$$W_k = \sum_{q=1}^k \sum_{x \in C_q} (x - c_q)(x - c_q)^T \quad (9)$$

where c_q is the centroid of the cluster q and c_o is the overall centroid of the datapoints. B_k calculates the quality of the separation between two clusters, i.e., higher the value of B_k , better it is. W_k calculates the distance of a datapoint (s) from the centroid of the cluster, and thus measures the cohesiveness or compactness of the datapoints within a cluster, i.e, smaller the value of W_k , better it is. Distribution-based clustering algorithms like K-Means tries to minimise the value of W_k . Thus according to the equation of C-H Index in Eq. 7, the higher the C-H index, the better is the clustering results. Because sum-of squared Euclidean distance is used to calculate the distance of the datapoints from their respective cluster centroids, C-H Index tends to favour convex clusters, which is a major drawback of using this metric as in Silhouette Coefficient.[6]

3.4.3 David-Bouldin index

David-Bouldin index is another metric used for evaluating clustering algorithms, which uses only the intrinsic features of the dataset and doesn't require any ground truth labels. David and Bouldin proposed R , a measure to calculate cluster similarity that satisfied the following conditions[52]:

- $R(S_i, S_j, M_{ij}) \geq 0$
- $R(S_i, S_j, M_{ij}) = R(S_j, S_i, M_{ij})$
- $R(S_i, S_j, M_{ij}) = 0$ iff $S_i = S_j$
- if $S_i = S_j$ and $M_{ij} < M_{ik}$, then $R(S_i, S_j, M_{ij}) > R(S_i, S_k, M_{ik})$
- if $M_{ij} = M_{ik}$ and $S_j > S_i$, then $R(S_i, S_j, M_{ij}) > R(S_i, S_k, M_{ik})$

where M_{ij} is the distance between the centroids of the clusters i and j and S_i and S_j are their respective dispersion measures. The conditions enforce that the cluster similarity measure R is non-negative and symmetrical in nature. It becomes 0 if and only if the dispersions of the two clusters disappear. If the distance M_{ij} increases even if S_i, S_j are constant, then the cluster similarity measure R decreases and *vice versa*. Also if M_{ij} remains constant and S_i, S_j increases, then R increases. Thus the David-Bouldin index was defined as the average of the similarities of each cluster as in Eq. 10, using the same notations as the authors.

$$\bar{R} = \frac{1}{C} \sum_{i=1}^C R_i \quad (10)$$

where C is the total number of clusters and R_i is the $\max_{j \neq i} R_{i,j}$. Thus, $\bar{R}$ or the David-Bouldin index is the average of the similarity measures R of each cluster with its most similar cluster. Thus, David-Bouldin index has a lower bound of 0, while a higher value means bad clustering. In other words, the less is the average similarity, the better is the clustering results. Because of this, it acts as a good measure to decide on the number of clusters that actually exists in the dataset. Thus, this index can be used to determine the number of clusters in distribution-based clustering algorithms like K-means. $R_{i,j}$ is more formally defined as in Eq. 11.

$$R_{i,j} = \frac{S_i + S_j}{M_{ij}} \quad (11)$$

The dispersion measure of a cluster i is defined as in Eq. 12

$$S_i = \left\{ \frac{1}{T_i} \sum_{j=1}^{T_i} \|X_j - A_i\|^q \right\}^{\frac{1}{q}} \quad (12)$$

where T_i are the number of datapoints in the cluster i , and X_j are the respective vector representations of the datapoints in the cluster i . For $q = 1$, The dispersion measure S_i is the average Euclidean distance of the vector representation of the datapoints from the centroid of the cluster i , while for $q = 2$, it is the standard

deviation of the metric about the datapoints in the cluster with respect to its centroid [52]. It is different as compared to the distance defined between the centroids of the clusters which are defined as in Eq. 13

$$M_{i,j} = \left\{ \sum_{k=1}^N \|a_{ki} - a_{kj}\|^p \right\}^{\frac{1}{p}} \quad (13)$$

where a_{ki} is the k -th component of the vector representation of a_i which is the centroid of the cluster i . $M_{i,j}$ is the Minkowski metric of the centroids of the clusters i and j , i.e., when $p = 1$, it is the Manhattan distance and when $p = 2$ it is the euclidean distance between the centroids. When both p and q are 2, then $R_{i,j}$ is the Fisher similarity measure between the two clusters[52]. But this metric too is not without its limitations. It is not robust to noise and outliers, thus giving an incorrect sense of good clustering. This index too is biased towards spherical clusters with similar sizes, thus making it inappropriate for many real world applications. Moreover, since it is a metric of the clustering results on a global level and gives no information about the individual clusters, even a single "too good" or "too bad" cluster can heavily influence the final result.

3.4.4 Density Based Cluster Validation Index

Thus, we could see how even the evaluation metrics using just the intrinsic features of the dataset are still not efficient in the presence of noise, outliers or non-convex clusters. As pointed out by Jain and Dubes, "without a strong effort in this direction, cluster analysis will remain a black art accessible only to those true believers who have experience and great courage" [25]. The fact that this statement holds even today, 35 years after he said it is even more astonishing. To tackle the problem of inefficient evaluation metrics for non-convex clusters or where the different clusters have a difference in size Moulavi *et. al.* proposed the Density Based Clustering Validation (DBCV) Index [38]. The authors have utilised the theory of Hartigan's model of density-contour trees[19]. The least dense region within a cluster and the most dense region outside a cluster is calculated with it which is analogous to computing the within cluster and between cluster density connectedness. As per the definition of Density Contour Trees[19] by Hartigan, density based clusters are regions of high density which are separated by low density regions. Having such model as a backdrop, it is expected that a good clustering algorithm which generate clusters such that the least dense region within a cluster is still more dense as compared to regions surrounding a cluster. Traditional intrinsic measure based evaluation metrics uses distance between the datapoints to compute cluster-variance and then incorporates it with the separation between the clusters to eventually compute the quality of the clustering algorithm. But this was not the target for DBCV. So they proposed a metric which was defined by densities instead of distances. DBCV wanted to take into account both the density and the shape of the cluster. So at first, the authors defined the concept of "all-points-core-distance" ($a_{ptscoredist}$). It was the inverse of the density of each object with respect to all other objects inside its cluster as given in Eq. 14.

$$a_{ptscoredist}(\mathbf{o}) = \left\{ \frac{\sum_{i=2}^{n_i} \left(\frac{1}{KNN(\mathbf{o}, i)} \right)^d}{n_i - 1} \right\}^{-\frac{1}{d}} \quad (14)$$

Using the above-mentioned equation, they defined a symmetric reachability distance which was then utilized to generate the Minimum Spanning Trees (MST) inside each clusters. Because the MSTs are built on the transformed space of symmetric reachability distances, it is capable of capturing both the density and shape of the cluster. On using these MSTs, they were able to find the least dense region within each of the clusters and the most dense regions between the pair of clusters. As per the definitions provided by the authors, $\mathbf{O} = o_1, o_2, \dots, o_n$ is the dataset containing n objects in the $\mathbb{R}^d$ feature space. $\mathbf{Dist}$ is an $n \times n$ matrix of pairwise distance $d(o_p, o_q)$ between the objects of the dataset, such that $o_p, o_q \in \mathbf{O}$. $KNN(\mathbf{o}, i)$ is the distance between the object $\mathbf{o}$ and its i^{th} nearest neighbor. $C = (C_i, N) 1 \leq i \leq l$ is the results returned by the clustering algorithms having l clusters and a set of N noise objects. n_i is the size of the i^{th} cluster and n_N is the number of objects in that particular cluster. To get an estimate of the density of an object within a cluster, traditional approaches use the notion of taking the inverse of the threshold distance required to find K objects within this threshold[19, 7]. But a drawback of this method would be that one would need to decide the value of the parameter K , which is not desirable. Moreover, with this approach, the density of the

object is dependent to its distance from only one other object i.e. the k^{th} nearest neighbor. This approach is not as robust as compared to considering more objects in its neighborhood as done in Gaussian kernel density estimate. Thus in order to propose a more robust density estimate, the authors proposed the use of a new parameterless core distance which can be elucidated as the inverse of a density estimate and thus can be used in defining the Mutual Reachability Distance (MRD). So now instead of using just one point in the cluster, all the points are taken into account such that the objects closer to it contribute to the density more as compared to the objects further away. We can see in Eq. 14, that the inverse of the KNN distances raised to the power of dimensions has been calculated such that closer objects gets a higher weight as compared to further objects. This effect can further be increased by using squared Euclidean distance instead of Euclidean distances. Also, as per the definition of MRD[33], the core distance of an object is compared to the distances of the object to other objects in the cluster. Hence, the core distance needs to be comparable to these distances. With the help of the following proof the authors have could rightfully conclude that for each object $\mathbf{o}$ in the cluster, the "all-points-core-distance" $a_{pts}coredist(\mathbf{o})$ lies between the second and the last nearest neighbor distance of the object[38] as in Eq. 15.

Proof. We have $\forall 2 \leq i \leq n$,

$$KNN(\mathbf{o}, 2) \leq KNN(\mathbf{o}, i) \implies \left(\frac{1}{KNN(\mathbf{o}, i)} \right)^d \leq \left\{ \frac{1}{KNN(\mathbf{o}, 2)} \right\}^d$$

Therefore,

$$\begin{aligned} \sum_{i=2}^n \left(\frac{1}{KNN(\mathbf{o}, i)} \right)^d &\leq (n-1) \left(\frac{1}{KNN(\mathbf{o}, 2)} \right)^d \\ \left((n-1) \frac{\frac{1}{KNN(\mathbf{o}, 2)}^d}{n-1} \right)^{-\frac{1}{d}} &\leq \left(\frac{\sum_{i=2}^n \left(\frac{1}{KNN(\mathbf{o}, i)} \right)^d}{n-1} \right)^{-\frac{1}{d}} \\ KNN(\mathbf{o}, 2) &\leq a_{pts}coredist(\mathbf{o}) \end{aligned}$$

The upper bound of the equation can be proved similarly giving rise to

$$a_{pts}coredist(\mathbf{o}) \leq KNN(\mathbf{o}, n)$$

□

$$KNN(\mathbf{o}, 2) \leq a_{pts}coredist(\mathbf{o}) \leq KNN(\mathbf{o}, n) \quad (15)$$

Moreover, the core distance is approximated to the distance of the object to its k^{th} nearest neighbor such that k is not too large. This means, that only a small neighborhood of the object is to be considered. Furthermore, let n objects be uniformly distributed random variables in a d -dimensional unit hypersphere. Then the core distance of object $\mathbf{o}$ is given by Eq. 16 which can be proved as following.

$$a_{pts}coredist(\mathbf{o}) = \ln(n)^{-\frac{1}{d}} \quad (16)$$

Proof. First the authors proved the following:

$$\forall 1 \leq i \leq j \leq n, \frac{KNN(\mathbf{o}, i)}{KNN(\mathbf{o}, j)} \approx \left(\frac{i-1}{j-1} \right)^{\frac{1}{d}} \quad (17)$$

where $KNN(\mathbf{o}, i)$ is the distance between the object $\mathbf{o}$ and its i_{th} nearest neighbor in the hypersphere. Now, the minimum volume of the hypersphere containing the i_{th} nearest neighbor of the object when the object is at the center of the hypersphere is given by Eq. 18

$$V_i \approx \frac{\pi^{\frac{d}{2}}}{\Gamma(\frac{d}{2} + 1)} r_i^d \quad (18)$$

where V_i is the minimum volume of the hypersphere with radius r_i that contains $KNN(\mathbf{o}, i)$. So, we get $KNN(\mathbf{o}, i) = r_i$. So in case of a unit hypersphere, $KNN(\mathbf{o}, i) \approx 1$. Also, because the objects are uniformly distributed, the ratio of the volume of the hyperspheres which contains the i_{th} and j_{th} nearest neighbors to the object $\mathbf{o}$ is $\frac{V_i}{V_j} \approx \frac{i-1}{j-1}$

$$\frac{V_i}{V_j} \approx \left(\frac{r_i}{r_j}\right)^d \quad (19)$$

$$\frac{r_i}{r_j} = \frac{KNN(\mathbf{o}, i)}{KNN(\mathbf{o}, j)} \approx \left(\frac{i-1}{j-1}\right)^{\frac{1}{d}} \quad (20)$$

From Eq. 17, we have,

$$\frac{I}{KNN(\mathbf{o}, i)} = \left(\frac{n-I}{i-I}\right)^{\frac{1}{d}} \frac{I}{KNN(\mathbf{o}, n)}$$

Therefore,

$$a_{ptscoredist}(\mathbf{o}) = \left(\frac{\sum_{i=2}^n \left(\frac{n-I}{i-I} \right)^{\frac{1}{d}} \frac{I}{KNN(\mathbf{o}, n)}}{n-I} \right)^{-\frac{1}{d}}$$

Since we have $KNN(\mathbf{o}, n) \approx 1$, therefore,

$$\begin{aligned} a_{ptscoredist}(\mathbf{o}) &= \left(\frac{\sum_{i=2}^n \left(\frac{n-I}{i-I} \right)^{\frac{1}{d}}}{n-I} \right)^{-\frac{1}{d}} \\ &= \left(\frac{\sum_{i=2}^n \left(\frac{n-I}{i-I} \right)}{n-I} \right)^{-\frac{1}{d}} \\ &= \left((n-I) \frac{\sum_{i=2}^n \left(\frac{1}{i-I} \right)}{n-I} \right)^{-\frac{1}{d}} \\ &= \left(\sum_{i=2}^n \left(\frac{1}{i-I} \right) \right)^{-\frac{1}{d}} \\ &= (\ln(n-I) + \gamma + \epsilon)^{-\frac{1}{d}} \\ &\approx \ln(n)^{-\frac{1}{d}} \end{aligned}$$

where $\sum_{i=1}^n \left(\frac{1}{i} \right)$ is a harmonic series. Furthermore, from Eq. 17,

$$\begin{aligned} \frac{KNN(\mathbf{o}, \frac{n}{\ln(n)})}{KNN(\mathbf{o}, n)} &\approx \left(\frac{\frac{n}{\ln(n)} - I}{n-I} \right)^{\frac{1}{d}} \\ &\approx \left(\frac{\frac{n}{\ln(n)}}{n} \right)^{\frac{1}{d}} \\ \frac{KNN(\mathbf{o}, \frac{n}{\ln(n)})}{KNN(\mathbf{o}, n)} &\approx \ln(n)^{-\frac{1}{d}} \end{aligned}$$

Since $KNN(\mathbf{o}, n) \approx 1$,

$$KNN\left(\mathbf{o}, \frac{n}{\ln(n)}\right) \approx \ln(n)^{-\frac{1}{d}}$$

Therefore, $a_{ptscoredist}(\mathbf{o}) \approx \ln(n)^{-\frac{1}{d}} \approx KNN(\mathbf{o}, j)$. where j is the closest natural number to $\frac{n}{\ln(n)}$. $\square$

This proves that the core distance of the object $\mathbf{o}$ is approximately equal to the same nearest neighbor distance and is independent of the dimension of the data space, if the objects are uniformly distributed. If the dataset doesn't have a uniform distribution, which is the case for most real world datasets, only the first property holds i.e. the density of the object doesn't depend on just one other object but on all objects in the cluster.

Now the $a_{ptscoredist}$ was used to compute the symmetric reachability distance which I would delve into deeper in the following paragraphs. As per the authors, the MRD between two objects $\mathbf{o}_i$ and $\mathbf{o}_j$ as in Eq. 21.

$$d_{mreach}(\mathbf{o}_i, \mathbf{o}_j) = \max\{a_{ptscoredist}(\mathbf{o}_i), a_{ptscoredist}(\mathbf{o}_j), d(\mathbf{o}_i, \mathbf{o}_j)\} \quad (21)$$

where $d(\mathbf{o}_i, \mathbf{o}_j)$ is the Euclidean distance between object $\mathbf{o}_i$ and $\mathbf{o}_j$. Once the MRD was computed, it could now be used to generate the mutual reachability graph where the objects are the different vertices and the graph and the MRD calculated above are the edge weights between the pair of vertices. Now with the set of objects $\mathbf{O}$ and the mutual reachability graph G , we get the MST of graph G as MST_{MRD} . So if we now quickly overview the process of DBCV, for a single cluster C_i having objects O , we first calculate the $a_{ptscoredist}$ of each object in the cluster. This is then used to compute the MRD between all pair of objects in the cluster. Depending on the MRD, the MST is generated. This process is then performed for all the clusters generated by the clustering algorithm, eventually giving rise to l MSTs if the clustering algorithms returns l clusters in total. Thus, we can notice that the number of MSTs increase linearly with the increase in the number of clusters. Also, the size of the tree depends on the number of objects in the clusters and thus can blow up exponentially in case of very large datasets having a lot of elements in each cluster.

From the MSTs generated above, the authors have crafted a new measure to validate the quality of the clustering algorithm depending on the density sparseness and density separation as explained below. The Density Sparseness of a Cluster (DSC) C_i can be visualized as the least dense region within the cluster and thus can be obtained from the maximum edge weight of the MST_{MRD} of that particular cluster. On the other hand, Density Separation of a Pair of Clusters (DSPC) C_i and C_j where $1 \leq i, j \leq l, i \neq j$ can be visualized as the region of highest density in between adjacent clusters and can be formulated as the minimum MRD between the objects of a cluster to the objects of its neighboring cluster. The DSC and the DSPC together then give the validity index of a cluster as in Eq. 22.

$$V_C(C_i) = \frac{\min_{1 \leq j \leq l, i \neq j} (DSPC(C_i, C_j)) - DSC(C_i)}{\max\left(\min_{1 \leq j \leq l, i \neq j} (DSPC(C_i, C_j)) - DSC(C_i)\right)} \quad (22)$$

Once we have the validity index of each cluster, the final DBCV score of the clustering algorithm can be defined as the weighted average of the validity index of all the clusters in C generated by the clustering algorithm as in Eq. 23. It is to be noted that even if noisy elements aren't explicitly present in the formula as a cluster, but it does however implicitly affects the overall score as the weighted average takes into consideration both the cardinality of individual clusters and well as the total number of elements in the dataset, which includes noise and outliers.

$$DBCVC(C) = \sum_{i=1}^l \frac{|C_i|}{|O|} V_C(C_i) \quad (23)$$

If a cluster has better density compactness within the cluster as compared to density separation in between adjacent clusters, then the validity index of the cluster would take a positive value. On the other hand, if

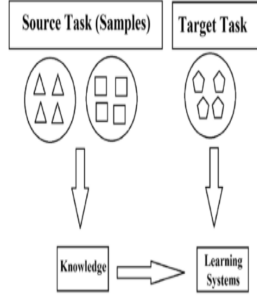


Figure 3.5.1: Model trained for a task in the similar domain is re-used for a new task.[23]

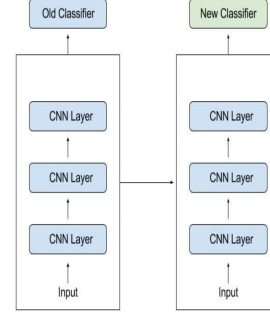


Figure 3.5.2: A model trained on a different task is adapted to be used for a new task.[59]

the density outside the cluster separating it from adjacent clusters is more than density compactedness inside the cluster than it would take a negative value. Thus, the DBCV score can vary in the range $[-1, 1]$ where greater DBCV score indicates to a better density-based clustering algorithm.

3.5 Transfer learning

Transfer learning is an optimization in machine learning techniques where a model trained for a particular task let's say task 1 is used for another task, task 2. It uses the knowledge learnt by the model in the previous task to be adapted for the new task. Transfer learning has its benefits in a multitude of applications because real world problems doesn't necessarily always have an abundant amount of labelled data which can be used to train a model from scratch[58]. It can be used for tasks which intend to solve different but related problems[58]. In other words, if the domain of the data for task 2 is similar to that of task 1, transfer learning can be used as shown in Fig.3.5.1. Several approaches have been adapted to use transfer learning. Let's say we want to solve task T_1 , but we don't have adequate data to train a deep neural network for it. But we have ample data to solve a similar task T_2 . So we can train a model for task T_2 and then use it for task T_1 . We might need to re-train only the later layers or all the layers of the model, but that heavily depends on the problem at hand. Another approach could be to use an existing pre-trained network. There are a lot of available open-source models which can be re-trained and re-used depending on the problem. Some of the popular models that are used as pre-trained models in transfer learning are Oxford VGG Model[48], Google Inception model[49], Microsoft ResNet model[20]. Another approach could be to use transfer learning for the purpose of feature extraction. Transfer learning has found a lot of application in the domain of computer vision and natural language processing because it requires a lot of data for training such complex models from scratch. In the domain of computer vision, we know that the early layers of a deep neural network focuses on learning the low level features like the detection of edges, while the middle layers focus on learning the mid level features like the detection of shapes. In transfer learning, these pre-trained early and middle layers could be used verbatim and only the latter layers which focus on the high level features could be re-trained for the new task as shown in fig3.5.2. Transfer learning has numerous advantages like significant reduction in the amount of training time[34], improvement in performance for some neural networks[41] and less reliance on a huge amount of data[14, 30, 61].

3.5.1 Definition

According to [58, 40], transfer learning can be formally defined as the following. A domain $\mathcal{D}$ of the input data, with the feature space $\mathcal{X}$ and a marginal probability distribution $P(X)$ such that $X = \{x_1, \dots, x_n\} \in \mathcal{X}$, where n is the number of feature vectors in X . In the given input data domain $\mathcal{D}$, the task $\mathcal{T}$ has a label space $\mathcal{Y}$ and a predictive function $f(\cdot)$ which is learned from the feature vector and label pairs (x_i, y_i) such that $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$ and $f(x)$ is the learner that predicts the label value for x . As per the above definition, the source domain is defined as $\mathcal{D}_S$ and the corresponding source task as $\mathcal{T}_S$. The target domain is defined as $\mathcal{D}_T$ and the corresponding target task as $\mathcal{T}_T$. Transfer learning aims at improving the target predictive function $f_T(\cdot)$ by using the related information from $\mathcal{D}_S$ and $\mathcal{T}_S$ where $\mathcal{D}_S \neq \mathcal{D}_T$ or $\mathcal{T}_S \neq \mathcal{T}_T$.

Transfer learning Settings	Related areas	Source Domain Labels	Target Domain Labels	Tasks
Inductive Transfer Learning	Multi-task learning	Available	Available	Regression, Classification
Transductive Transfer Learning	Self-taught Learning	Unavailable	Available	
	Domain Adaptation, Simple Selection bias, Co-variate Shift	Available	Unavailable	
Unsupervised Transfer Learning		Unavailable	Unavailable	Clustering, Dimensionality Reduction

Table 1: Different settings of transfer learning [40]

Since $\mathcal{D}_S = \{X_S, P(X_S)\}$ and $\mathcal{D}_T = \{X_T, P(X_T)\}$, here the condition $\mathcal{D}_S \neq \mathcal{D}_T$ means that $X_S \neq X_T$ and/or $P(X_S) \neq P(X_T)$. The scenario where $X_S \neq X_T$ in the context of transfer learning is defined as heterogeneous transfer learning. The scenario where $X_S = X_T$ is defined as homogeneous transfer learning. The scenario where $P(X_S) \neq P(X_T)$ means the source and the target data are from different domains, i.e., they belong to different marginal distributions. Transfer learning algorithms are not expected to give optimal results in this case. Referring back to the definition of transfer learning, when $\mathcal{T}_S \neq \mathcal{T}_T$ and it was defined that $\mathcal{T} = \{(Y), P(Y|X)\}$, this scenario is possible when $\mathcal{Y}_S \neq \mathcal{Y}_T$ and/or $P(Y_S|X_S) \neq P(Y_T|X_T)$. The scenario where $P(Y_S|X_S) \neq P(Y_T|X_T)$ implies that the conditional probability distributions between the source and target domains are different. The case $\mathcal{Y}_S \neq \mathcal{Y}_T$ mean the label space of the source and target domain are different.[58]

3.5.2 Categories

As per the definition of transfer learning mentioned above, it can be broadly categorized into 3 groups: inductive transfer learning, transductive transfer learning and unsupervised transfer learning as mentioned in [40]. Depending on the availability of labels of the source and target tasks, these 3 categories are possible as shown in Table 1.

Regardless of whether the source and target domains are the same or not, the target task in an inductive transfer learning situation is distinct from the source task. In this instance, in order to generate an objective predictive function $f_T(\cdot)$ for usage in the target domain, some labeled data from the target domain are needed. Furthermore, inductive transfer learning can be broken down into two groups depending on the availability of source and target domain labels. When a lot of labeled data is available for the source domain, it is similar to multi-task learning. But the difference lies in the fact that the goal of inductive transfer learning is to attain good results in the target task only by using the knowledge obtained from the source task while multi-task learning endeavors to learn both the target and source tasks concurrently[40]. If the labels of the source data are not available, then it is similar to the self-taught learning as proposed by [45]. In such a scenario, the label space of the source data and the target data can vary which means that the knowledge obtained from the source domain cannot be utilised directly. This implies that basically the labels of the source domain are not available.[40]

On the other hand, in transductive transfer learning, the target task and the source task are same, but the source domain and the target domain are different. Depending on the similarity of the feature space and the marginal probability distribution between the source and the target domain, transductive transfer learning too can be classified into two groups. One scenario can be that the feature space of the source domain and that of the target domain are different i.e. $X_S \neq X_T$. Again, the feature space of the two domain can be same, but the input data's marginal probability distributions are different i.e. $P(X_S) \neq P(X_T)$. Similar assumptions underlie the latter instance of the transductive transfer learning, which is related to sample selection bias [60], covariate shift [47], and domain adaptation for knowledge transfer in text categorization [12, 60].[40] Lastly, in unsupervised transfer learning, the target task is different from the source task as it was in inductive transfer learning, but the labels of neither the source domain nor the target domain

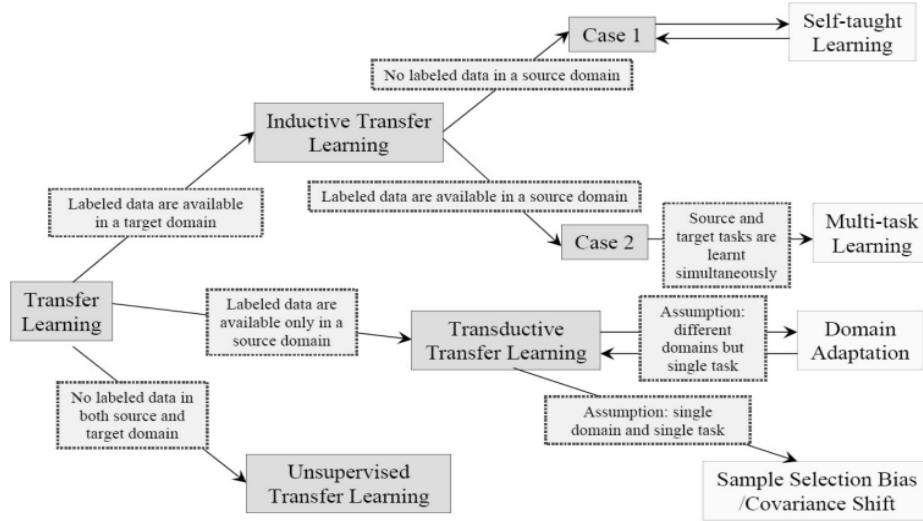


Figure 3.5.3: Overview of the different types of transfer learning.[40]

is available. the goal of unsupervised transfer learning is to solve unsupervised learning problems in the target domain like density estimation, dimensionality reduction and clustering[11, 56]. The categorisation of transfer learning has been summarised in Fig.3.5.3

3.5.3 Relevant Algorithms

3.6 Optimal Transport Problem

3.7 Sinkhorn-Knopp Algorithm

3.8 Dimensionality reduction

3.9 Network generation process with PQ-Net++

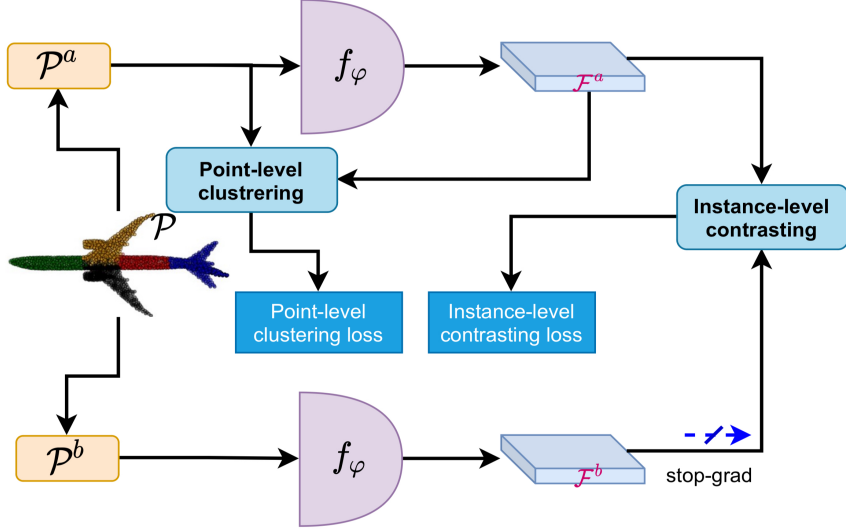


Figure 4.1.1: The Conclu model pipeline including the two modules: point-level clustering and instance-level contrasting.[36]

4 Method

This chapter gives the details of the networks and clustering algorithms that were examined. The unsupervised feature representation learning approach proposed by Guofeng Mei *et. al.*[36] was adapted to be used in our scenario. The authors proposed an unsupervised learning approach, ConClu that learns both the local features and well as the global features of the point clouds. It was suitable to be used in our scenario, as it would require no point-level annotation of the dataset, which was unavailable in our case. To do so, it performed point-level clustering of the points clouds to generate semantically related regions within the point cloud and performed instance-level contrasting to make the network robust to its global appearance which would be explained in detail in the following subsections.[36]

The primary motivation behind using an unsupervised learning methodology is that annotation of point clouds can be really challenging. To really focus on the point level features of a point cloud, one would require a high number of points in the point cloud. And labelling such high number of points in each point cloud in large real world datasets can be really expensive and severely inefficient. Unsupervised learning facilitates high quality feature representation without the need for humans in the loop. Moreover, sporadic and non-uniform distribution of the point clouds can also add further challenges in point-level annotation of the point clouds. Thus, in the absence of such annotations, it is impossible to get supervised feature representations for computer vision tasks.

4.1 Architecture

Analysing how humans perceive an object, they do not focus merely on the individual points but rather semantically related points that form a group and act as the fundamental block for the entire object as a whole. The global features focus on the entire shape of the object while the local features attains to the more detailed information in the different parts of the object. In this work, we have 3D point cloud $\mathbf{P} = \{\mathbf{p}_i \in \mathbb{R}^3 | i = 1, 2, 3, \dots, N\}$ where N is the number of points in each point cloud and $\mathbf{p}_i$ is each point in the point cloud represented in 3D cartesian coordinate system. The point cloud $\mathbf{P}$ is fed to an encoder backbone f_ϕ which generates a point-wise feature matrix $\mathbf{F} = \{f_{p_i}\}_{i=1}^N$ where f_{p_i} is the point-wise feature vector. The objective was to train a feature encoder f_ϕ , PointNet in our case, having parameters ϕ in an unsupervised learning paradigm. The entire pipeline of the process is elaborated in Fig.4.1.1. As we can see, it has two parts: the point-level clustering for learning the local features and the instance-level contrasting for learning the global features. Before we go into a detailed analysis of each module, here is

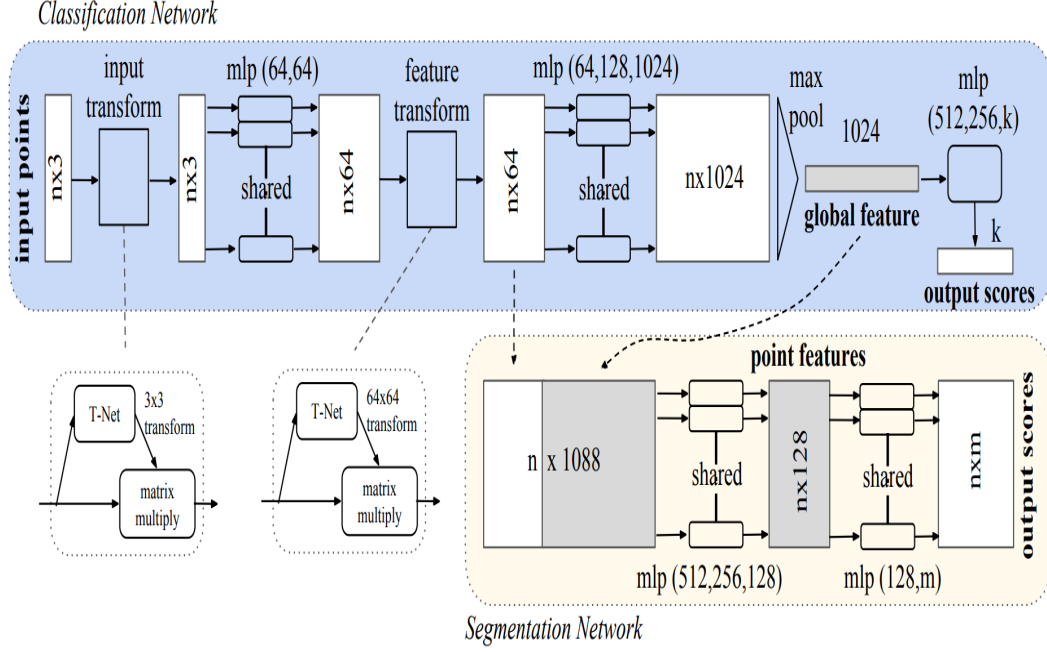


Figure 4.2.1: The PointNet Architecture.[44]

a brief overview of the entire process. The augmented views $\mathbf{P}^a$, and $\mathbf{P}^b$ are obtained for each point cloud $\mathbf{P}$ and are fed to the encoder f_ϕ such that it generates the feature matrices $\mathbf{F}^a$, and $\mathbf{F}^b$ respectively. Thus, the inputs to the point-level clustering are $\mathbf{P}^a$ and $\mathbf{F}^a$. It is to be noted here that the feature encoder f_ϕ shares the weights between the two augmented views. f_ϕ is trained in a way such that it minimises the point-level clustering and the instance-level contrasting loss together. It is also to be noted here, that the encoder receives the gradient only from the top branch. After the training is completed, both the losses are cast aside and only the encoder is needed for further tasks.

4.2 Encoder Backbone

Keeping in line with [36], two feature representation learning backbones were used for the experiments, the PointNet [44] and the Dynamic Graph Convolutional Neural Network (DGCNN) [55] backbone. The mechanism and the architecture of the feature extraction backbones have been elaborated further in the following sections.

4.2.1 PointNet Autoencoder

For the purpose of weight sharing and other kernel optimization tricks, traditional convolution networks need extremely regular formats for the input data as prevalent in Red Green Blue (RGB) or depth images and 3D voxels. But point clouds are not regular data structures. It is just a set of 3D points which is permutation invariant to its constituent points. Or in other words, the order in which the individual points of a point cloud are processed is inconsequential. In [44], the authors suggested an end-to-end architecture that accepts the point clouds as input and generates the class label of the entire object or a class label for each of the individual points in the point cloud. The PointNet architecture is shown in Fig. 4.2.1 It consists of three important modules: a symmetric function to aggregate information, a module that integrates the local and the global features, an alignment network that orients the points of the point cloud and their features. A further detailed overview of each of the building blocks are given in the following passages.

Symmetry function The points in the point cloud are an unordered set of points, which means the order in which they appear is irrelevant. Therefore, to make the network invariant to permutation, the following

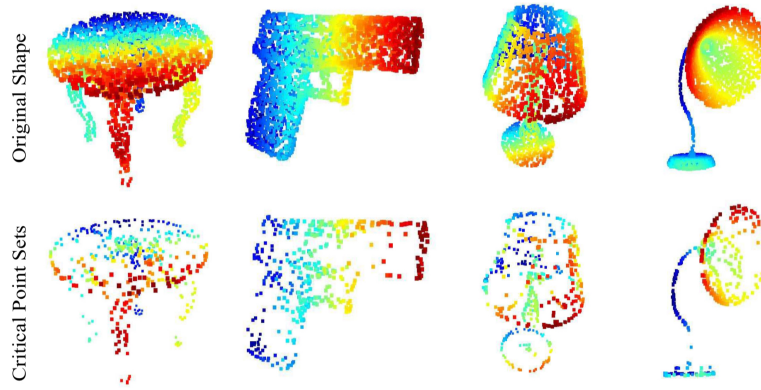


Figure 4.2.2: Critical points set and the original shape.[44]

approaches could be adopted.

- The input points could be sorted in canonical manner.
- To treat the input points like sequential data and train a Recurrent Neural Network (RNN). But in order to counter the effect, the RNN needs to be trained with all possible set of permutations
- The data from each point could be combined using a symmetric function. Binary addition and multiplication are some examples of symmetric functions.

Sorting the points isn't an efficient solution because in high dimension space, there isn't actually any stable ordering w.r.t point perturbations. Contradiction is a simple way to demonstrate that. If such an ordering existed, it would mean there is a bijection mapping between the higher dimension space and a 1d real line. For this ordering to be stable, the bijection mapping would need to maintain spatial proximity even if the dimensions decreased in the event of point perturbations. This is not feasible in most cases. As a result, sorting only partially fixes the ordering problem, and it brings additional challenges for the network to learn a consistent bijection mapping between the input and the output. The second approach was to treat the points as a sequential data and training an RNN with different randomly permuted input sequence in order to make it invariant to the order in which the input appears. But in [54], it was proved that the order indeed matters and cannot be overlooked completely. Moreover, even if an RNN is robust to the order of the input with small length(e.g. dozens), it suffers from scalability issues. It can't be scaled to inputs with thousands of elements as it is in point clouds. Hence, the idea proposed by the authors in [44] was to devise a symmetric function on the transformed set of points to approximate the general function defined on the points as in Eq. 24

$$f(\{x_1, \dots, x_n\}) \sim g(h(x_1), \dots, h(x_n)) \quad (24)$$

where $f : 2_{\mathbb{R}^N \rightarrow \mathbb{R}}, \mathbb{R}^N \rightarrow \mathbb{R}^K$ and $g : \underbrace{\mathbb{R}^K \times \dots \times \mathbb{R}^K}_n \rightarrow \mathbb{R}$ is the symmetric function. Here h is a multi-layer

perceptron network and g is a combination of a single variable function and a max pooling function. Even if this module is fairly simple, it has some of very important implications. If C_S is the set of critical points, the points which contributed to the max pooled feature, the PointNet architecture was seen to be quite robust to the non-critical points. In other words, discarding the non-critical points didn't lead to any loss of information about the global shape of the object as shown in Fig. 4.2.2.

If needed we can add theoretical analysis here

Local and Global Information Aggregation The output of the symmetry function is a vector $[f_1, \dots, f_K]$ which is the global feature representation of the point cloud. But, to get point-wise segmentation, information about both the local features and the global features are necessary. The local features are learnt by the network in the *Segmentation Network* module as shown in Fig. 4.2.1. Once the global feature vector of the point cloud is calculated, it is concatenated with each of the point features and is fed into the network. This enables the network to learn new point-level features when it is informed about both the local and the

global information. Because of this module, the network is able to predict the class probability of individual point based on both the local geometry of the object as well as the semantic region it belongs to on a global-level.[44]

Joint Alignment Network The network should be invariant to certain geometric transformations like rigid body transformations (e.g. rotation, translation). A trivial solution to this issue is aligning every input set to a canonical order prior to feature extraction. But this would require an additional layers for aligning all input sets. On the contrary, the authors in [44] utilized a "mini-network" (*T-net*) as shown in Fig. 4.2.1. This predicts a transformation matrix which is then applied to the coordinates of the points in the point cloud. The mini-network too takes after the main network and is made up of point independent feature extraction module, max pooling layers and the fully connected layers. To align the point-features from different point cloud sets, another alignment network could be introduced. It would predict the transformation matrix required for the alignment. But that would significantly increase the difficulty of optimization since the transformation matrix has much higher dimensions as compared to the spatial transformation matrix. To circumvent this issue, a regularization term was used in the softmax training loss. The constraint enforced in this case was that the feature transformation matrix is to be approximated by an orthogonal matrix as given in Eq. 25

$$L_{reg} = \|I - AA^T\|_F^2 \quad (25)$$

where $\|\cdot\|_F$ is the Frobenius-norm, I is the Identity matrix, and A is the feature alignment matrix predicted by the mini-network. The feature transformation matrix has been approximated by an orthogonal matrix because orthogonal transformations ensure that no information about the input has been lost.

4.2.2 Dynamic Graph Convolutional Neural Network

Another more general feature encoder was used in this work. It was seen that the PointNet architecture treated each point of the point cloud independently. The model was made invariant to the different possible permutations by applying a symmetric function to aggregate the features. But a limitation to this method is that it doesn't take into consideration the topology of the points. It ignores the connectedness in between the points and thus cannot learn the local features. This limitation was addressed by Y. Wang *et al.* in [55]. The authors proposed the EdgeConv operation which was able to learn the local features of the point cloud while still meeting the requirement of being permutation invariant. EdgeConv generated the edge features which were capable of capturing the topological information of the points i.e. the connection between a point and its neighbors. In other words, the EdgeConv operation computes a local graph for each of the points in the point cloud which was capable of learning the edge embeddings for the edges in the graph. So the model groups points based on both the Euclidean distance between the points and also how they were semantically related. The EdgeConv is a plug-and-play module that would be used combined into any existing model architecture to further enhance its performance. The approach has been further elaborated in the following passages.

Unlike PointNet, the DGCNN network captures the topological information by generating a local neighborhood graph and then applying an operation analogous to the convolution operation on the edges connecting a pair of points. It is in line to the methodology used in graph neural networks. But unlike a traditional graph CNN, the local neighborhood graph is not fixed in this case. On the contrary, it is updated after every layer in the network. This is because the k-nearest neighbors of a point doesn't remain the same after every layer of the network. This happens as the k-nearest neighbors are computed not based on the Euclidean distance between the points, but rather of the proximity of the points based on the learnt embeddings. In other words, the proximity of the points in the feature space are different from the proximity of the input points in the Euclidean space. This is responsible for the non-local flow of information through the point cloud.

Edge Convolution In this work, we have 3D point cloud $P = \{p_i \in \mathbb{R}^F | i = 1, 2, 3, \dots, N\}$ where N is the number of points in each point cloud and p_i is each point in the point cloud represented in 3D cartesian coordinate system where $p_i = (x_i, y_i, z_i)$. In this work, $F = 3$, but it is also possible to include additional information about the coordinates such as surface normals, color, etc. like most traditional deep neural

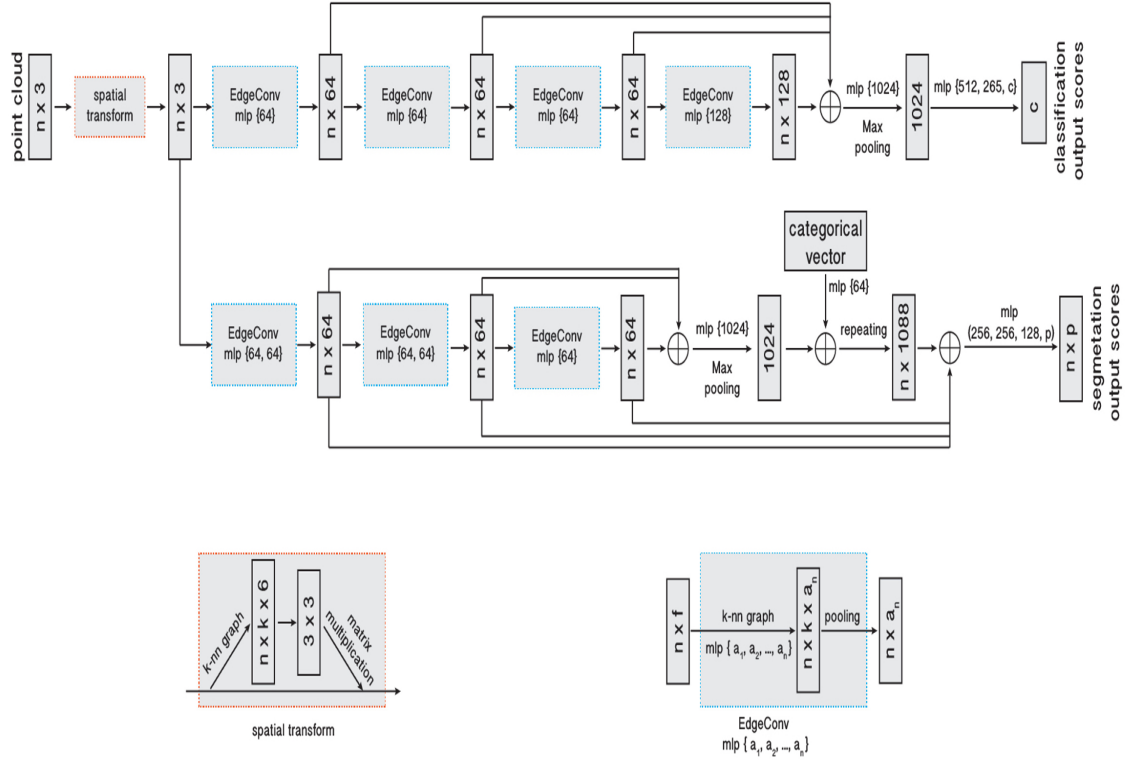


Figure 4.2.3: The DGCNN Architecture.[55]

networks, each layer acts upon the outputs of the previous layer. So generally speaking, F is the dimension of the features of a particular layer in the network. The architecture of the model used in this work is shown in Fig. 4.2.3. The top branch is used for the classification of the point cloud and the bottom branch is for the point-wise segmentation task. The classification task requires the n points in point cloud as input. k is pre-defined as the number of nearest neighbors that are to be considered for generating the edge features. The EdgeConv layer computes an edge feature set of size k for each point p_i in the point cloud. An aggregation operation is performed on each of these sets to get the EdgeConv response for the respective points. A global aggregation function is applied on the output of the last EdgeConv layer to generate a 1D global feature descriptor. This is used to compute the classification score for c classes. The point-wise segmentation branch further extends the classification branch by concatenating the 1D global descriptor with all EdgeConv responses (acts as local descriptors) for each point in the point cloud. Thus, it generates a point-wise classification score for p semantic classes that it can belong to. $\oplus$ is the concatenation operation. The spatial transform block is the module used to align the input point cloud to a canonical space. It is done by applying an estimated 3×3 transformation matrix. The coordinates of each of the points in the point cloud are concatenated with the difference in the coordinates with the k neighboring points giving rise to a tensor. This tensor is used in computing above-mentioned transformation matrix. The input point cloud of shape $n \times f$, here $f = 3$ is fed to the EdgeConv block which computes the edge features for each of the points in the point cloud. An Multi Layer Perceptron (MLP) is used for this purpose where the number of neurons in a layer is defined as $\{a_1, a_2, \dots, a_n\}$. the output of this block is a tensor with shape $n \times a_n$ after performing a pooling operation on the neighboring edge features.

A directed graph $G = (V, E)$ which captures the topology of the point cloud where $V = \{p_i | i = \{1, 2, \dots, n\}\}$ is the set of vertices and $E \subset V \times V$ is the set of edges. The graph G is the k -Nearest Neighbor (k -NN) graph of $\mathcal{P}$ in $\mathbb{R}^F$. the graph has self-loops which means, each point in the point cloud has an edge to itself. The edge features computed by the EdgeConv block was defined as $e_{ij} = h_\theta(p_i, p_j)$ where $h_\theta : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}^{F'}$ is a non-linear function with a set of trainable parameters θ . The EdgeConv operation is defined as a

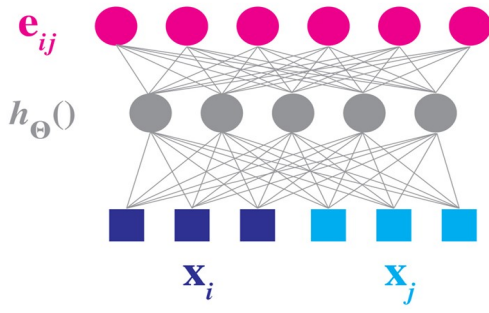


Figure 4.2.4: The Computation of edge feature e_{ij} from a pair of points p_i and p_j . Here, h_θ is a fully connected layer and the weights associated with it are the trainable parameters θ . [55]

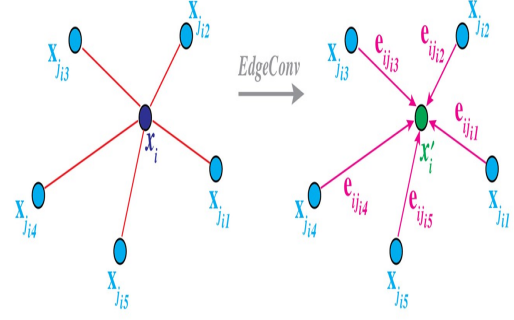


Figure 4.2.5: The EdgeConv operation. The output of EdgeConv is computed by aggregating the features associated with all the edges that originate from each connected vertex. [55]

symmetric aggregation operation $\square$ which is applied on a channel wise manner. Summation (Σ) or max are some of the examples of aggregation functions. It is applied on the edge features associated with all the edges that originate from each connected vertex. Therefore, the output of the EdgeConv layer for i -th vertex is given by Eq. 26

$$p'_i = \square_{j:(i,j) \in E} h_\theta(p_i, p_j) \quad (26)$$

drawing similarity with convolution on images, p_i is the central pixel and $\{p_j : (i, j) \in E\}$ is a patch around the central pixel as shown in Fig. 4.2.5. Therefore, for a point cloud with F dimensions and n points in it, the EdgeConv layer generates a point cloud with F' dimensions and n points in it.

Deciding the non-linear function h_θ and the symmetric aggregation function $\square$ plays a pivotal role in the deciding the properties of the EdgeConv layer. To further illustrate on this point, if $p_1, \dots, p_n$ are the points in the point cloud and the graph G encapsulates the topological information of patches of fixed size around each central pixel, then if $\theta_m \cdot p_j$ is chosen as the non-linear function and sum (Σ) is chosen as the aggregation operation, then it would behave as a standard convolution operation as in eq. 27

$$p'_{im} = \sum_{j:(i,j) \in E} \theta_m \cdot p_j \quad (27)$$

where $\theta = (\theta_1, \theta_2, \dots, \theta_M)$ are the weights of M different filters used in the convolution operation. Each entry in θ_m has the dimensionality as p_i and $\cdot$ is the Euclidean inner product. Another variation of h could be

$$h_\theta(p_i, p_j) = h_\theta(p_i). \quad (28)$$

Here only the global embedding is encoding is taken into consideration and the non-linear function is unaware of the local features. This happens in PointNet and thus it could be considered as a special case of EdgeConv. Another variation of h as adapted by Atzmon *et al.* in [2] is

$$h_\theta(p_i, p_j) = h_\theta(p_j). \quad (29)$$

and

$$p'_{im} = \sum_{j \in V} (h_\theta(p_j)) g(u(p_i, p_j)). \quad (30)$$

Here g is a Gaussian kernel and u calculates the pairwise Euclidean distance between the points in the point cloud. Another variation of h could be to only capture the local features and ignore the global features based on the fact that the global shape is a collection of the local patches as shown in the following

$$h_\theta(p_i, p_j) = h_\theta(p_j - p_i). \quad (31)$$

The final variation of h which was utilized by the authors in [55] is an asymmetric edge function as shown in eq. 32

$$h_{\theta}(p_i, p_j) = \bar{h}_{\theta}(p_i, p_j - p_i). \quad (32)$$

This explicitly takes into account the global feature information as in Eq.28 captured by the coordinates of the central pixel of the patch p_i and the information of the local neighborhood as in Eq. 31 and is captured by $p_j - p_i$. the EdgeConv operator was defined as

$$e'_{ijm} = \text{ReLU}(\theta_m \cdot (p_j - p_i) + \phi_m \cdot p_i). \quad (33)$$

This operator could be implemented as a shared MLP and then performing a max-pooling operation as

$$p'_{im} = \max_{j:(i,j) \in E} e'_{ijm}, \quad (34)$$

where $\theta = (\theta_1, \theta_2, \dots, \theta_M)$ are the weights of M different filters used in the convolution operation.

Dynamic Graph Update A crucial point in this network was to regenerate the graph using k-NN in the feature space after each layer. This is the main difference of this network as compared to traditional graph CNN for which there is a fixed graph for all the layers of the network. Because the graph was updated after every layer, the architecture gets its name the DGCNN. With dynamically updating the graph after each layer, the receptive field increases to as large as diameter of the point cloud, even if the points in the point cloud are sparse. Thus after each layer a different graph is computed $G^{(l)} = (V^{(l)}, E^{(l)})$, where $G^{(l)}$ is a graph in the l -th layer with vertices $V^{(l)}$ and edges $E^{(l)}$ which can be denoted as $(i, j_{i1}), \dots, (i, j_{ik_i})$ such that $p_{j_{i1}}^{(l)}, \dots, p_{j_{ik_i}}^{(l)}$ are the k-nearest points closest to point p_i at layer l . In other words, the model constructs a different graph for each point in the point cloud after every layer where the nearest points and the edges emerging from a vertex (point in the point cloud in the case) changes at every layer. A pairwise distance matrix in the feature space is used to calculate the k-nearest points of each point in the point cloud. As a result of this, the network has the following properties,

- *Invariant to permutations.* The output of an EdgeConv layer is given by

$$p'_i = \max_{j:(i,j) \in E} h_{\theta}(p_i, p_j). \quad (35)$$

Because of the usage of a max operator which is symmetric in nature, the output of the layer p'_i is permutation invariant to the order in which p_i appears in the input point cloud. Other functions which are symmetric (eg. $\sum$) in nature can also be applied here instead of max. the global max pooling operator used to aggregate point features is a symmetric function and thus gives permutation invariant results.

- *Invariant to translation.* The EdgeConv operator satisfies "partial" translation invariance. If a translation T is applied to the points p_i and p_j of the point cloud $\mathcal{P}$, then applying translation to Eq. 33, the translated point cloud would be

$$\begin{aligned} e'_{ijm} &= \theta_m \cdot (p_j + T - (p_i + T)) + \phi_m \cdot (p_i + T), \\ &= \theta_m \cdot (p_j - p_i) + \phi_m \cdot (p_i + T). \end{aligned} \quad (36)$$

Therefore, if $\phi_m = 0$, then EdgeConv operator is completely translation invariant. But this however restricts the model to recognize an object depending on only the unordered set of patches, i.e. it disregards the orientation and the exact location of the patches. On the other hand, with both $p_j - p_i$ and p_i , the model considers both the local orientation of the patches as well as the global information about the shape of the object.

4.3 Point-level Clustering Module

Now, the model already has the feature representation from the feature encoder backbone (eg. PointNet or DGCNN). The next step is the point-level clustering module. Fig.4.3.1 gives a detailed visualisation

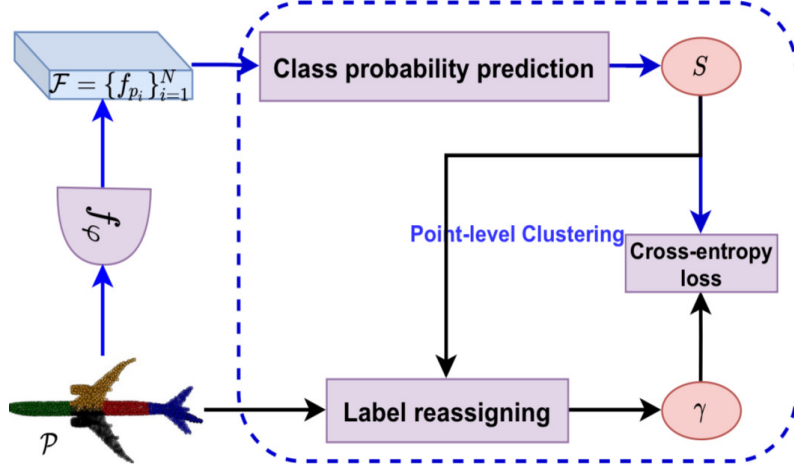


Figure 4.3.1: The architecture of the point-level clustering based unsupervised feature learning.[36]

of the point-level clustering module. We can see that it is made up of 2 entites, the "class probability prediction" function and the "label reassigning". The point-level clustering module is analogous to the semantic segmentation task. Each point p_i of the point cloud $\mathcal{P}$ is assigned to one of the J predetermined possible subgroups or semantic categories. Essentially, it is achieved by processing each of the point cloud $\mathbf{P}$ followed by a point-level class probability prediction operator that yields a matrix for class probability $\mathbf{S} = \{s_{ij} \in [0, 1]\}_{i,j}^{N,J}$. The label reassigning operator thus takes the point cloud $\mathbf{P}$ and the class probability matrix $\mathbf{S}$ and generates a pseudo-label matrix $\gamma = \{\gamma_{ij} \in \{0, 1\}\}_{i,j}^{N,J}$. The weights for the encoder f_ϕ are learnt such that it minimizes the average cross-entropy loss between the pseudo-label γ and the predicted class probability $\mathbf{S}$. The average cross entropy loss is calculated by the formula given in Eq.37.

$$\epsilon(\gamma, \mathbf{S}) = -\frac{1}{N} \langle \gamma, \log \mathbf{S} \rangle = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^J \gamma_{ij} \log s_{ij} \quad (37)$$

But to train a network with a cross-entropy loss, the ground truth labels of the points in the point cloud are required. Since that is not available, one needs to assign the label γ automatically, which is explained in details in the following passage.

4.3.1 Class Probability prediction

As mentioned in the last passage, the output of the encoder f_ϕ is fed as input to a classification head ϕ_α and it generates a logit score for each of the feature vector as shown in Fig.4.3.2. The vector for the logit score is given by $\mathbf{g}_i = (g_{i1}, g_{i2}, \dots, g_{iJ})$, where $\mathbf{g}_i = \gamma_\alpha(f_{p_i})$. The classification head has 2 fully connected layers where each layer is a linear layer followed by a batch normalization. The leakyReLU activation function is used in all the layers except the last layer. As the output of the last layer we get N vectors for each of the N points in the point cloud where each of the output vector has J dimensions, the number of predetermined subcategories each point cloud is to be segmented into. Thus each entry of the feature vector is the probability of that point belonging to j -th category. Thus, the logit score matrix $\mathbf{G} = \{g_{ij}\}_{i,j}^{N,J}$ has the dimension $N \times J$. The prediction that the point p_i belongs to the category j is computed by the application of a row-wise softmax operation on $\mathbf{G}$ given by Eq.38

$$s_{ij} = \frac{\exp(g_{ij})}{\sum_{l=1}^J \exp(g_{il})} \quad (38)$$

4.3.2 Label reassigning

In order to calculate the labels shown in Fig.4.3.3, one needs to the representative members of each of the semantic subgroups. It can be done by selecting the cluster mean or cluster centers of each of these

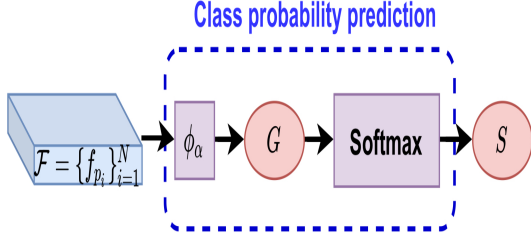


Figure 4.3.2: Architecture of class probability prediction.[36]

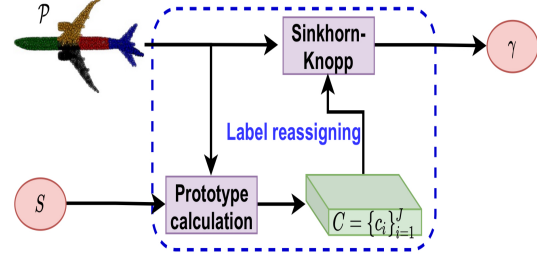


Figure 4.3.3: Architecture of label reassigning [36]

subgroups as the representative member of the cluster. The class probability matrix S can be inferred as the soft assignment of each point p_i in the point cloud to J semantic partitions. So the softly weighted mean of each of these partitions can be calculated as in Eq.39

$$c_j = \frac{1}{\sum_{i=1}^N s_{ij}} \sum_{i=1}^N s_{ij} \mathbf{p}_i, j = 1, 2, \dots, J \quad (39)$$

or in another way $C = \{c_j\}_{j=1}^J$ is a matrix with dimensions $J \times 3$. In a completely unsupervised learning setup, $\gamma \in [0, 1]$ can be visualized as the posterior probability of a p_i belong to the j -th partition. γ was optimized in the same method as in Eq.38. This results in a degenerate solution where each point is assigned to a single category. Thus, two assumptions were very pivotal for the reassigning of the labels. Firstly, the points in the point cloud ought to be partitioned into "equally-sized" partitions. In other words it can be formulated as $\sum_{i=1}^N \gamma_{ij} = \frac{N}{J}$. This is extremely importance in order to pretend all the points in the point cloud to be assigned to a single partition. Secondly, since the idea of using cluster centers as representative members of the partitions was inspired from the k-means algorithm, it means that if a point p_i of the point cloud belongs to the j^* -th partition, it means that the distance from p_i to c_j should be the shortest as compared to the distance of p_i to all other cluster centers. In other words $\|\mathbf{p}_i - c_{j^*}\|_2 \leq \|\mathbf{p}_i - c_j\|_2, j \neq j^*$. This is also equivalent to $\min_{\gamma} \sum_{i=1}^N \sum_{j=1}^J \gamma_{ij} \|\mathbf{p}_i - c_j\|_2^2$. This objective function is optimised to get γ . As per the rules of probability, the sum of all possible outcomes of an event is equal to 1, therefore, we get $\sum_{j=1}^J \gamma_{ij} = 1$. Thus, we can formulate the matrix of joint probability Γ as $\Gamma = \frac{\gamma}{N}$ which has the elements $\Gamma_{ij} = \frac{\gamma_{ij}}{N}$. The distance matrix $D = \{d_{ij}\}_{i,j=1}^{N,J}$ has dimensions $N \times J$ where each element d_{ij} is defined as $d_{ij} = \|\mathbf{p}_i - c_j\|_2^2$. In accordance to the assumptions stated above, the objective function D could be minimized with respect to Γ as in Eq.40

$$\begin{aligned} \min_{\Gamma} \langle \Gamma, D \rangle, \\ s.t., \Gamma^T \mathbf{1}_N = \frac{1}{J} \mathbf{1}_J, \Gamma \mathbf{1}_J = \frac{1}{N} \mathbf{1}_N \end{aligned} \quad (40)$$

where $\mathbf{1}_J$ is the vector of ones having dimension J . The above condition enforces that each of the cluster representative is selected atleast $\frac{N}{J}$ times for each point cloud and $\sum_{j=1}^J \gamma_{ij} = 1$. The objective function as defined in Eq.40 is a typical case of optimal transport problem[43]. Such problems can be solved by the Sinkhorn-Knopp algorithm[10]. In other words, it means solving the entropic regularized objective function in Eq. 41

$$\begin{aligned} \min_{\Gamma} \langle \Gamma, D \rangle - \epsilon H(\Gamma), \\ s.t., \Gamma^T \mathbf{1}_N = \frac{1}{J} \mathbf{1}_J, \Gamma \mathbf{1}_J = \frac{1}{N} \mathbf{1}_N, \end{aligned} \quad (41)$$

where, $H(\Gamma) = \langle \Gamma, \log \Gamma - 1 \rangle$ is the entropy of Γ and ϵ is the regularization parameter. As per [10], on solving Eq. 41, the normalized exponential matrix in Eq.42 is attained.

$$\Gamma = \text{diag}(\mu) \exp(D/\epsilon) \text{diag}(\nu) \quad (42)$$

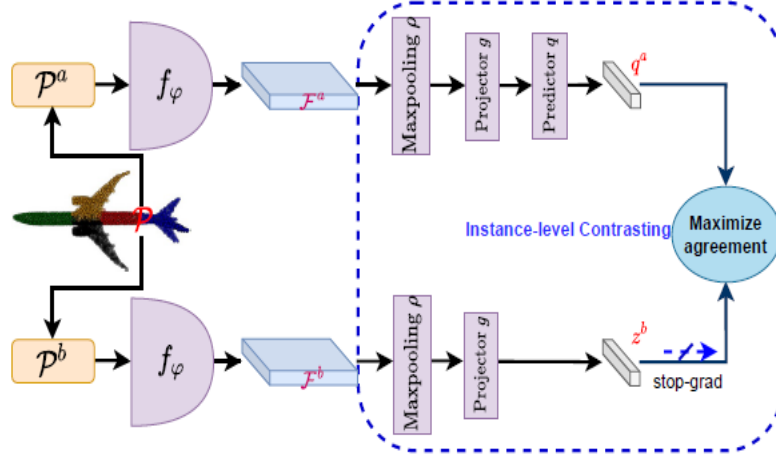


Figure 4.4.1: The architecture of the instance-level Contrasting.[36]

where μ and $\mathbf{v}$ are renormalized vectors in $\mathbb{R}^N$ and $\mathbb{R}^J$ respectively. These vectors were computed on utilizing the iterative Sinkhorn-Knopp algorithm[10] with the constraints $\mu = \frac{1}{N} \mathbf{1}_N$ and $\mathbf{v} = \frac{1}{J} \mathbf{1}_J$. The pseudo-code for the Sinkhorn-Knopp algorithm is elaborated in Algorithm 3. In the experiments, it was observed that 10 iterations were sufficient for a decent performance. Thus the overall point-level clustering can be encapsulated by an Expectation Maximization (EM) like algorithm. The network learns the parameters f_ϕ and ϕ_α by optimization of Eq.37 and get a label reassigning matrix γ by solving the optimization problem in Eq. 42 with respect to Γ . It is executed by performing the following steps in repetition.

- Step 1: representation learning. Having the current probability matrix γ , the weights of the model are updated by optimizing Eq. 37 with respect to the parameters f_ϕ and ϕ_α . It is in line with the supervised case where the model uses the cross entropy loss for classification.
- Step 2: label reassigning. On having the updated set of f_ϕ and ϕ_α , Γ is calculated by using Eq. 42. The posterior probability matrix is then computed $\gamma = N \cdot \Gamma$

Thus each update step requires a single matrix-vector multiplication with has the time complexity of $O(N \times J)$ which means the computation time increases linearly with the increase in the number of points in the point cloud. Thus scalability of this method is quite high, and the process remains relatively quick even for a million samples in the dataset. Moreover, the orthogonal regularization was introduced to circumvent the issue of having the same output vector for all cluster representatives. It is computed by

$$\mathcal{L}_{orth}(C) = \|C_*^T C_* - I\|_1, \quad (43)$$

where $\|\cdot\|_1$ is the l_1 -norm or the Manhattan distance and $C_* = [\frac{c_1}{\|c_1\|_2}, \frac{c_2}{\|c_2\|_2}, \dots, \frac{c_J}{\|c_J\|_2}]$. The pseudo-code for the point-level clustering module is elaborated in 1

4.4 Instance-level Contrasting Module

This module plays a crucial role in learning the global features of the point clouds. The architecture of which is shown in Fig. 4.4.1. The network is fed with two augmented views $\mathcal{P}^a, \mathcal{P}^b$ of the same point cloud $\mathcal{P}$. The feature representation obtained as the output of the encoder backbone f_ϕ , are fed to a MaxPooling layer ρ , the output of which is fed to a projection MLP head g [8]. The encoder backbone f_ϕ and the projection head g shares the weights for both the augmented views. The output of the projection head of one of the augmented views (the top branch) is fed to the prediction head q . The purpose of this unit is to transform the output of one of the augmented views to match the other. The operation is applied to only one augmented view to make the network asymmetric [18]. In other words, the output vectors can be realized as $\mathbf{q}^a \triangleq \mathbf{q}(g(\rho(\mathcal{F}^a)))$ and $\mathbf{z}^b \triangleq g(\rho(\mathcal{F}^b))$ where $\mathcal{F}^a = f_\phi(\mathcal{P}^a)$ and $\mathcal{F}^b = f_\phi(\mathcal{P}^b)$. Keeping in line

with [8], stop-gradient (stopgrad) was applied on the second branch z^b to resist the network from generating a constant mapping without using negative samples for contrastive learning. The network is constructed such that the similarity between q^a and z^b is maximized. Mthematically speaking, the mean-squared error between the l_2 -normalized prediction q^a and projection z_b is minimized as shown in Eq. 44.

$$\mathcal{D}(q^a, z^b) \triangleq \left\| \frac{q^a}{\|q^a\|_2} - \frac{z^b}{\|z^b\|_2} \right\|_2^2 = 2 - \frac{2q^{aT}z^b}{\|q^a\|_2 \cdot \|z^b\|_2}, \quad (44)$$

In other words, it is the negative cosine similarity, up to a scale of 2. Since, stopgrad is enforced on z^b , Eq. 44 is modified by the following

$$\mathcal{D}(q^a, \text{stopgrad}(z^b)) \quad (45)$$

which means, z^b acts as a constant vector in this equation. Keeping in line with [8], a global symmetrized loss was defined as in Eq. 46

$$\mathcal{L}_{global} = \mathcal{D}(q^a, \text{stopgrad}(z^b)) + \mathcal{D}(q^b, \text{stopgrad}(z^a)) \quad (46)$$

The minimum value the above equation can attain is 0. This is because, in the first term of the equation, the encoder backbone q^a gets no gradient flow from z^b . But it gets gradients from q^b in the second term. In the same way, in the second term of the equation, the encoder backbone q^b gets no gradient flow from z^a . But it gets gradients from q^a in the first term. if the stopgrad mechanism was not applied, then in spite of having a loss of zero during training, the representations learnt by the network would be futile i.e., all the point clouds would be mapped to the same feature representation. or in other words, the network would crumble to constant mapping [8].

4.5 Loss function

The objective of the network is to simultaneously minimize both the point-level clustering loss and the instance level contrasting loss. If the two augmented views of the point cloud $\mathcal{P}$ are $\mathcal{P}^a$ and $\mathcal{P}^b$, then the point-level clustering loss can be calculated by Eq. 47

$$\mathcal{L}_{local} = \varepsilon(\gamma^a, S^a) + \varepsilon(\gamma^b, S^b) + \eta(\mathcal{L}_{orth}(C^a) + \mathcal{L}_{orth}(C^b)) \quad (47)$$

where $\eta > 0$ is a regularization parameter. The value of η is set to in this work. Here γ^a is the posterior class probability of the point belonging to one of the subcategories, S^a is the predicted class probability and C^a is the normalized center of the augmented view of the point cloud $\mathcal{P}^a$. In the same way, the corresponding terms for the other augmented view $\mathcal{P}^b$ are γ^b , S^b and C^b . Therefore, the total loss could be calculated as the linear combination of $\mathcal{L}^{global}$ and $\mathcal{L}^{local}$ as

decide value of eta

$$\mathcal{L}_{total} = \mathcal{L}_{local} + \mathcal{L}_{global} \quad (48)$$

4.6 Clustering Algorithm

Once

4.7 Suitable Algorithm and Adjustments

Algorithm 1 Pseudo-code for point-level clustering

Input: $\{\mathcal{P}\}$: is a set of 3D point clouds with N number of points in it; K : number of epochs.

Output: backbone f_ϕ .

```

1: for  $i$  in range  $(0, K)$  do
2:    $\mathcal{L} = 0$ 
3:   for  $\mathcal{P} \in \{\mathcal{P}\}$  do
4:     # Computes Class probability
5:      $S = \text{softmax}(\phi_\alpha(f_\phi(\mathcal{P})))$ 
6:     # Computes cluster representatives of partitions
7:      $C = \left\{ \frac{1}{\sum_{i=1}^N s_{ij}} \sum_{i=1}^N s_{ij} \mathbf{p}_i \right\}_{j=1}^N$ 
8:     # Computes mean-squared error
9:      $D = \left\{ \|\mathbf{p}_i - c_j\|_2^2 \right\}_{i,j}^{N,J}$ 
10:    # Computes posterior probability matrix
11:     $\gamma = \text{sinkhorn}(\text{stopgrad}(D), 1e-3, 20)$ 
12:    # Computes Loss Function
13:     $\mathcal{L} = \varepsilon(\gamma, S) + \eta \mathcal{L}_{\text{orth}}(C)$ 
14:  end for
15:  # Updates backbone, projector and predictor
16:   $f_\phi, \phi_\alpha \leftarrow \text{optimize}(\frac{\mathcal{L}}{N})$ 
17: end for
18: return  $\mathbf{f}_\phi$ 

```

Algorithm 2 Pseudo-code for instance-level clustering

Input: $\{\mathcal{P}\}$: is a set of 3D point clouds with N number of points in it; K : number of epochs.

Output: backbone f_ϕ .

```

1: for  $i$  in range  $(0, K)$  do
2:    $\mathcal{L} = 0$ 
3:   for  $\mathcal{P} \in \{\mathcal{P}\}$  do
4:     # Generate random augmentations
5:      $\mathcal{P}^a = \text{aug}(\mathcal{P})$ 
6:      $\mathcal{P}^b = \text{aug}(\mathcal{P})$ 
7:     # Compute projections
8:      $z^a = g(\rho(f_\phi(\mathcal{P}^a)))$ 
9:      $z^b = g(\rho(f_\phi(\mathcal{P}^b)))$ 
10:    # Compute predictions
11:     $q^a, q^b = q(z^a), q(z^b)$ 
12:    # Calculate loss
13:     $\mathcal{L} += \mathcal{D}(q^a, \text{stopgrad}(z^b)) + \mathcal{D}(q^b, \text{stopgrad}(z^a))$ 
14:  end for
15:  # Updates backbone, projector and predictor
16:   $f_\phi, g, q \leftarrow \text{optimize}(\frac{\mathcal{L}}{N})$ 
17: end for
18: return  $\mathbf{f}_\phi$ 

```

Algorithm 3 Pseudo-code for Sinkhorn-Knopp algorithm

```
def sinkhorn( $D, \varepsilon, niters = 3$ )
1:  $\Gamma = \exp(D/\varepsilon)^T$ 
2:  $\gamma/ = \text{sum}(\Gamma)$ 
3:  $N, J = \Gamma.\text{shape}$ 
4:  $u, r, c = \text{zeros}(N), \text{ones}(J)/J, \text{ones}(N)/N$ 
5: for  $\text{inrange}(0, niters)$  do
6:    $u = \text{sum}(\Gamma, \text{dim} = 1)$ 
7:    $\Gamma^* = (r/u).\text{unsqueeze}(1)$ 
8:    $\Gamma^* = (c/\text{sum}(\Gamma, \text{dim} = 0)).\text{unsqueeze}(0)$ 
9: end for
10: return  $\Gamma$ 
```

4.8 Whatever new Concept we propose

4.9 Source and Target Objects

4.10 Data Generation

4.11 Important Aspects

5 Application of algorithms

6 Experiments

7 Implementation in a Bin Picking Cell

8 Conclusions

8.1 Discussions

8.2 Limitations

8.3 Future Scope

9 Summary

List of Acronyms

CNN Convolutional Neural Network

FCN Fully Connected Network

DGCNN Dynamic Graph Convolutional Neural Network

ReLU Rectified Linear Unit

ML Machine Learning

AI Artificial Intelligence

DBSCAN Density-Based Spatial Clustering of Applications with Noise

HDBSCAN Hierarchical Density-Based Spatial Clustering of Applications with Noise

VAE Variational Autoencoders

KL Kullback–Leibler divergence

DBCV Density Based Clustering Validation

MST Minimum Spanning Trees

MRD Mutual Reachability Distance

DSC Density Sparseness of a Cluster

DSPC Density Separation of a Pair of Clusters

EM Expectation Maximization

MLP Multi Layer Perceptron

stopgrad stop-gradient

RGB Red Green Blue

RNN Recurrent Neural Network

k-NN k-Nearest Neighbor

List of Figures

3.2.1 Architecture of an undercomplete autoencoder.[37]	4
3.2.2 Architecture of a single layer sparse encoder. The hidden nodes in bright yellow are active, while the ones in light yellow are set to zero, hence inactive[3]	4
3.2.3 Difference between a vanilla autoencoder and a VAE.[51]	5
3.2.4 A generic siamese network.[1]	6
3.2.5 Before (left) and after (right) minimizing triplet loss function[50]	6
3.2.6 Minibatches in offline triplet mining. The green boxes are positive datapoints, the blue boxes are anchor datapoints and the red boxes are negative datapoints.	7
3.2.7 Triplets constructed only within the minibatch. There are P object classes in the minibatch. The green boxes are positive datapoints, the blue box is the anchor datapoint and the red boxes are negative datapoints.	7
3.3.1 DBSCAN can find non-linearly separable non-convex clusters.[13]	8
3.3.2 Difference between K-Means and K-Medoids.[15]	9
3.3.3 A dendrogram in hierarchical clustering.[22]	9
3.5.1 Model trained for a task in the similar domain is re-used for a new task.[23]	16
3.5.2 A model trained on a different task is adapted to be used for a new task.[59]	16
3.5.3 Overview of the different types of transfer learning.[40]	18
4.1.1 The Conclu model pipeline including the two modules: point -level clustering and instance-level contrasting.[36]	19
4.2.1 The PointNet Architecture.[44]	20
4.2.2 Critical points set and the original shape.[44]	21
4.2.3 The DGCNN Architecture.[55]	23
4.2.4 The Computation of edge feature e_{ij} from a pair of points p_i and p_j . Here, h_θ is a fully connected layer and the weights associated with it are the trainable parameters θ . [55]	24
4.2.5 The EdgeConv operation. The output of EdgeConv is computed by aggregating the features associated with all the edges that originate from each connected vertex.[55]	24
4.3.1 The architecture of the point-level clustering based unsupervised feature learning.[36]	26
4.3.2 Architecture of class probability prediction.[36]	27
4.3.3 Architecture of label reassigning [36]	27
4.4.1 The architecture of the instance-level Contrasting.[36]	28

List of Tables

1 Different settings of transfer learning [40] 17

List of Symbols

Bibliography

- [1] *A Comprehensive Guide to Siamese Neural Networks*. <https://medium.com/@rinkinag24/a-comprehensive-guide-to-siamese-neural-networks-3358658c0513>.
- [2] Matan Atzmon, Haggai Maron, and Yaron Lipman. “Point convolutional neural networks by extension operators”. In: *arXiv preprint arXiv:1803.10091* (2018).
- [3] *Autoencoder*. <https://en.wikipedia.org/wiki/Autoencoder>.
- [4] Vassileios Balntas et al. “Learning local feature descriptors with triplets and shallow convolutional neural networks.” In: *Bmvc*. Vol. 1. 2. 2016, p. 3.
- [5] Jane Bromley et al. “Signature verification using a siamese time delay neural network”. In: *Advances in neural information processing systems* 6 (1993).
- [6] *Calinski-Harabasz Index*. https://en.wikipedia.org/wiki/Calinski-Harabasz_index.
- [7] Ricardo JGB Campello, Davoud Moulavi, and Joerg Sander. “A simpler and more accurate AUTO-HDS framework for clustering and visualization of biological data”. In: *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 9.6 (2012), pp. 1850–1852.
- [8] Xinlei Chen and Kaiming He. “Exploring simple siamese representation learning”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 15750–15758.
- [9] *Convolutional Neural Networks, Explained*. <https://towardsdatascience.com/convolutional-neural-networks-explained-9cc5188c4939>.
- [10] Marco Cuturi. “Sinkhorn distances: Lightspeed computation of optimal transport”. In: *Advances in neural information processing systems* 26 (2013).
- [11] Wenyuan Dai et al. “Self-taught clustering”. In: *Proceedings of the 25th international conference on Machine learning*. 2008, pp. 200–207.
- [12] Hal Daume III and Daniel Marcu. “Domain adaptation for statistical classifiers”. In: *Journal of artificial Intelligence research* 26 (2006), pp. 101–126.
- [13] *DBSCAN Wikipedia*. <https://en.wikipedia.org/wiki/DBSCAN>.
- [14] Lixin Duan, Dong Xu, and Ivor Tsang. “Learning with augmented features for heterogeneous domain adaptation”. In: *arXiv preprint arXiv:1206.4660* (2012).
- [15] Alireza Entezami, Hassan Sarmadi, and Behzad Saeedi Razavi. “An innovative hybrid strategy for structural health monitoring by modal flexibility and clustering methods”. In: *Journal of Civil Structural Health Monitoring* 10.5 (2020), pp. 845–859.
- [16] Martin Ester et al. “A density-based algorithm for discovering clusters in large spatial databases with noise”. In: *kdd*. Vol. 96. 34. 1996, pp. 226–231.
- [17] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [18] Jean-Bastien Grill et al. “Bootstrap your own latent-a new approach to self-supervised learning”. In: *Advances in neural information processing systems* 33 (2020), pp. 21271–21284.
- [19] John A Hartigan. *Clustering algorithms*. John Wiley & Sons, Inc., 1975.
- [20] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [21] Alexander Hermans, Lucas Beyer, and Bastian Leibe. “In defense of the triplet loss for person re-identification”. In: *arXiv preprint arXiv:1703.07737* (2017).
- [22] *Hierarchical Clustering*. <https://harshsharma1091996.medium.com/hierarchical-clustering-996745fe656b>.
- [23] Asmaul Hosna et al. “Transfer learning: a friendly introduction”. In: *Journal of Big Data* 9.1 (2022), p. 102.

- [24] *Intuitively Understanding Variational Autoencoders*. <https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf>.
- [25] Anil K Jain and Richard C Dubes. *Algorithms for clustering data*. Prentice-Hall, Inc., 1988.
- [26] Xin Jin and Jiawei Han. “K-Means Clustering”. In: *Encyclopedia of Machine Learning*. Ed. by Claude Sammut and Geoffrey I. Webb. Boston, MA: Springer US, 2010, pp. 563–564. ISBN: 978-0-387-30164-8. DOI: 10.1007/978-0-387-30164-8_425. URL: https://doi.org/10.1007/978-0-387-30164-8_425.
- [27] Xin Jin and Jiawei Han. “K-Medoids Clustering”. In: *Encyclopedia of Machine Learning*. Ed. by Claude Sammut and Geoffrey I. Webb. Boston, MA: Springer US, 2010, pp. 564–565. ISBN: 978-0-387-30164-8. DOI: 10.1007/978-0-387-30164-8_426. URL: https://doi.org/10.1007/978-0-387-30164-8_426.
- [28] Diederik P Kingma, Max Welling, et al. “An introduction to variational autoencoders”. In: *Foundations and Trends® in Machine Learning* 12.4 (2019), pp. 307–392.
- [29] Gregory Koch, Richard Zemel, Ruslan Salakhutdinov, et al. “Siamese neural networks for one-shot image recognition”. In: *ICML deep learning workshop*. Vol. 2. 1. Lille. 2015.
- [30] Brian Kulis, Kate Saenko, and Trevor Darrell. “What you saw is not what you get: Domain adaptation using asymmetric kernel transforms”. In: *CVPR 2011*. IEEE. 2011, pp. 1785–1792.
- [31] Solomon Kullback and Richard A Leibler. “On information and sufficiency”. In: *The annals of mathematical statistics* 22.1 (1951), pp. 79–86.
- [32] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [33] Levi Lelis and Jörg Sander. “Semi-supervised density-based clustering”. In: *2009 Ninth IEEE International Conference on Data Mining*. IEEE. 2009, pp. 842–847.
- [34] Tong Liu et al. “Exploring transfer learning to reduce training overhead of hpc data in machine learning”. In: *2019 IEEE International Conference on Networking, Architecture and Storage (NAS)*. IEEE. 2019, pp. 1–7.
- [35] Claudia Malzer and Marcus Baum. “A hybrid approach to hierarchical density-based cluster selection”. In: *2020 IEEE international conference on multisensor fusion and integration for intelligent systems (MFI)*. IEEE. 2020, pp. 223–228.
- [36] Guofeng Mei et al. “Unsupervised learning on 3d point clouds by clustering and contrasting”. In: *arXiv preprint arXiv:2202.02543* (2022).
- [37] Sumit Misra et al. “An autoencoder based model for detecting fraudulent credit card transaction”. In: *Procedia Computer Science* 167 (2020), pp. 254–262.
- [38] Davoud Moulavi et al. “Density-based clustering validation”. In: *Proceedings of the 2014 SIAM international conference on data mining*. SIAM. 2014, pp. 839–847.
- [39] Andrew Ng et al. “Sparse autoencoder”. In: *CS294A Lecture notes* 72.2011 (2011), pp. 1–19.
- [40] Sinno Jialin Pan and Qiang Yang. “A survey on transfer learning”. In: *IEEE Transactions on knowledge and data engineering* 22.10 (2009), pp. 1345–1359.
- [41] Weike Pan et al. “Transfer learning to predict missing ratings via heterogeneous user feedbacks”. In: *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain*. 2011, p. 2318.
- [42] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [43] Gabriel Peyré, Marco Cuturi, et al. “Computational optimal transport: With applications to data science”. In: *Foundations and Trends® in Machine Learning* 11.5-6 (2019), pp. 355–607.
- [44] Charles R Qi et al. “Pointnet: Deep learning on point sets for 3d classification and segmentation”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 652–660.

- [45] Rajat Raina et al. “Self-taught learning: transfer learning from unlabeled data”. In: *Proceedings of the 24th international conference on Machine learning*. 2007, pp. 759–766.
- [46] Salah Rifai et al. “Contractive auto-encoders: Explicit invariance during feature extraction”. In: *Proceedings of the 28th international conference on machine learning*. 2011, pp. 833–840.
- [47] Hidetoshi Shimodaira. “Improving predictive inference under covariate shift by weighting the log-likelihood function”. In: *Journal of statistical planning and inference* 90.2 (2000), pp. 227–244.
- [48] Karen Simonyan and Andrew Zisserman. “Very deep convolutional networks for large-scale image recognition”. In: *arXiv preprint arXiv:1409.1556* (2014).
- [49] Christian Szegedy et al. “Going deeper with convolutions”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 1–9.
- [50] *Triplet Loss and Siamese Neural Networks*. <https://medium.com/@enoshshr/triplet-loss-and-siamese-neural-networks-5d363fdeba9b>.
- [51] *Understanding Variational Autoencoders(VAEs)*. <https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73>.
- [52] Alberto A Vergani and Elisabetta Binaghi. “A soft davies-bouldin separation measure”. In: *2018 IEEE international conference on fuzzy systems (FUZZ-IEEE)*. IEEE. 2018, pp. 1–8.
- [53] Pascal Vincent et al. “Extracting and composing robust features with denoising autoencoders”. In: *Proceedings of the 25th international conference on Machine learning*. 2008, pp. 1096–1103.
- [54] Oriol Vinyals, Samy Bengio, and Manjunath Kudlur. “Order matters: Sequence to sequence for sets”. In: *arXiv preprint arXiv:1511.06391* (2015).
- [55] Yue Wang et al. “Dynamic graph cnn for learning on point clouds”. In: *ACM Transactions on Graphics (tog)* 38.5 (2019), pp. 1–12.
- [56] Zheng Wang, Yangqiu Song, and Changshui Zhang. “Transferred dimensionality reduction”. In: *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2008, Antwerp, Belgium, September 15-19, 2008, Proceedings, Part II* 19. Springer. 2008, pp. 550–565.
- [57] Kilian Q Weinberger and Lawrence K Saul. “Distance metric learning for large margin nearest neighbor classification.” In: *Journal of machine learning research* 10.2 (2009).
- [58] Karl Weiss, Taghi M Khoshgoftaar, and DingDing Wang. “A survey of transfer learning”. In: *Journal of Big data* 3.1 (2016), pp. 1–40.
- [59] *What Is Transfer Learning? Exploring the Popular Deep Learning Approach*. <https://builtin.com/data-science/transfer-learning>.
- [60] Bianca Zadrozny. “Learning and evaluating classifiers under sample selection bias”. In: *Proceedings of the twenty-first international conference on Machine learning*. 2004, p. 114.
- [61] Yin Zhu et al. “Heterogeneous transfer learning for image classification”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 25. 1. 2011, pp. 1304–1309.